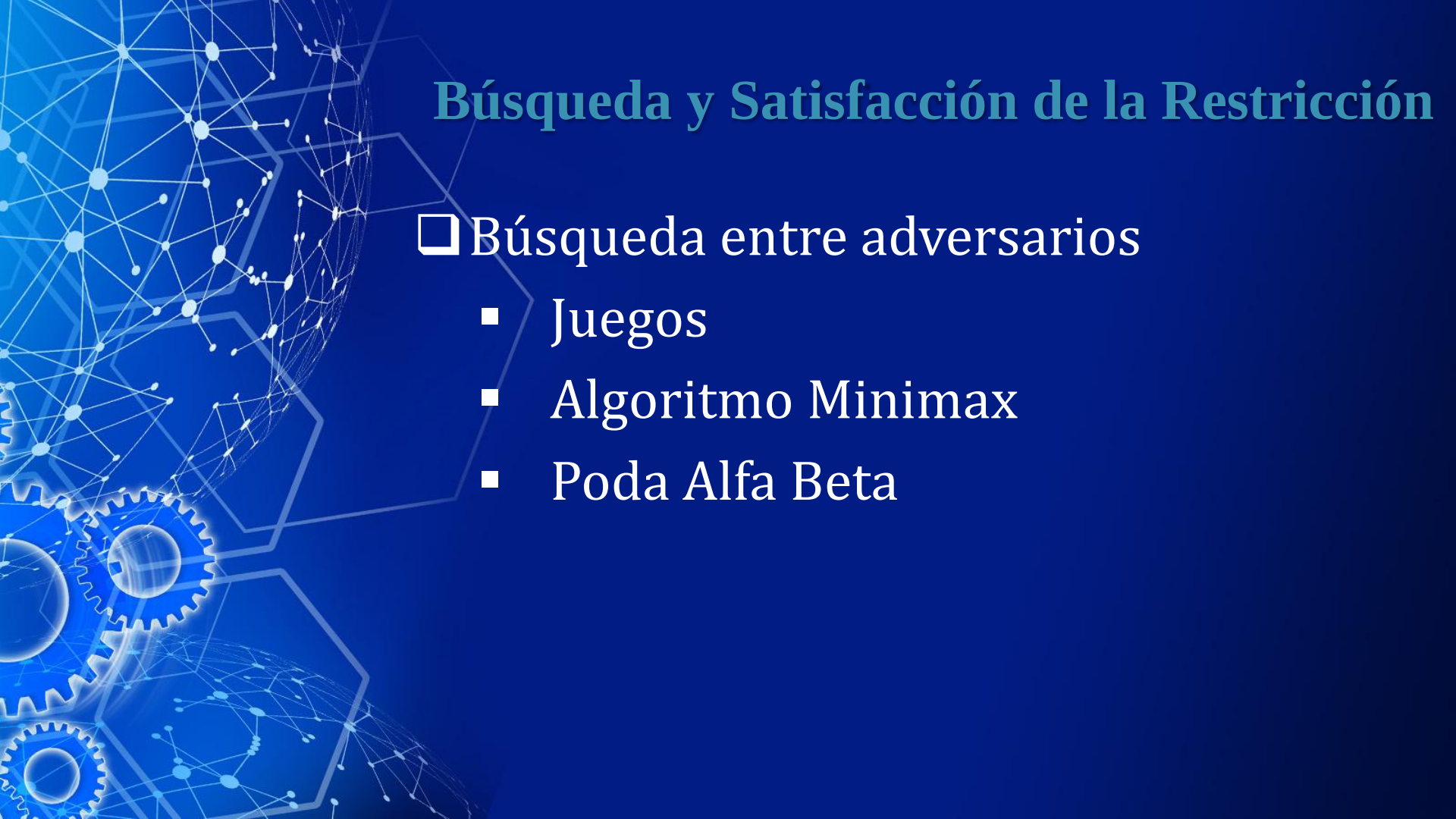




Búsqueda y Satisfacción de la Restricción

- Búsqueda entre adversarios
 - Juegos
 - Algoritmo Minimax
 - Poda Alfa Beta



Búsqueda de adversarios

- ❑ En los **entornos multiagente (cooperativos o competitivos)**, cualquier agente tiene que considerar las acciones de otros agentes.
- ❑ La imprevisibilidad de estos otros agentes puede introducir muchas **contingencias** en el proceso de resolución de problemas.
- ❑ Los entornos competitivos, en los cuales los objetivos de los agentes están en conflicto, dan ocasión a problemas de **búsqueda entre adversarios**, a menudo conocidos como **juegos**.

Búsqueda de adversarios

- ❑ La **teoría matemática de juegos**, una rama de la economía, ve a cualquier entorno multiagente como un juego.
- ❑ Los “juegos” que se tratan en IA son una clase más especializada:

	Juegos deterministas	Juegos de azar
Con información perfecta	Ajedrez, damas, Go, Othello	Backgammon, Monopoly
Con información imperfecta	barquitos	Bridge, poker, scrabble



Juegos : Introducción

¿Los juegos son fáciles de resolver?

Juego	Estados
3 en raya	$9! = 362280$
Conecta 4	10^{13}
Damas	10^{18}
Ajedrez	10^{47}
Go	10^{170}

❑ Los juegos, como el mundo real, requieren la capacidad de tomar *alguna* decisión (la jugada) cuando es in-factible calcular la decisión *óptima*.

Juego perfecto

- ☐ Dos jugadores (Min, Max)
- ☐ Movimientos intercalados
- ☐ Suma cero (la ganancia de uno es la pérdida del otro).
- ☐ Información perfecta (ambos jugadores tienen acceso a toda la información sobre el estado del juego: no se ocultan información el uno al otro).
- ☐ No interviene el azar (p.ej. dados).

- ☐ Ejemplos:
Nim, Grundy, 3 en raya, conecta-4, damas, ajedrez...

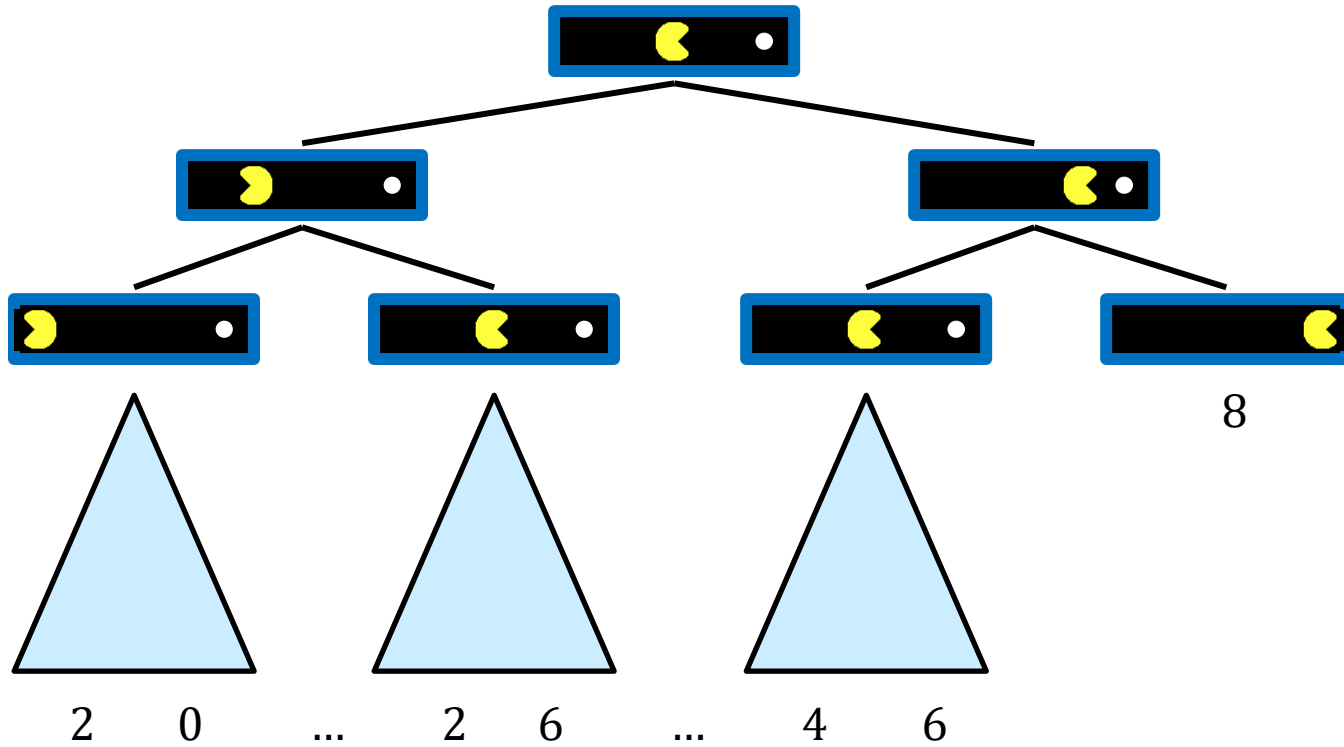


Juegos: Representación

- ❑ Puede ser definido como un problema de espacio de estados
 - **Estado:** situaciones por las que puede pasar el juego
 - **Estado inicial:** (comienzo del juego)
 - **Estados finales:** (final del juego)
 - En los estados finales, especificaremos qué jugador gana
 - **Movimientos:** operadores que se aplican a los estados, cambiando las situaciones de juego
 - **Función de utilidad:** valoración (respecto de MAX) basada en las reglas del juego, que se asigna a un estado final y a un turno.

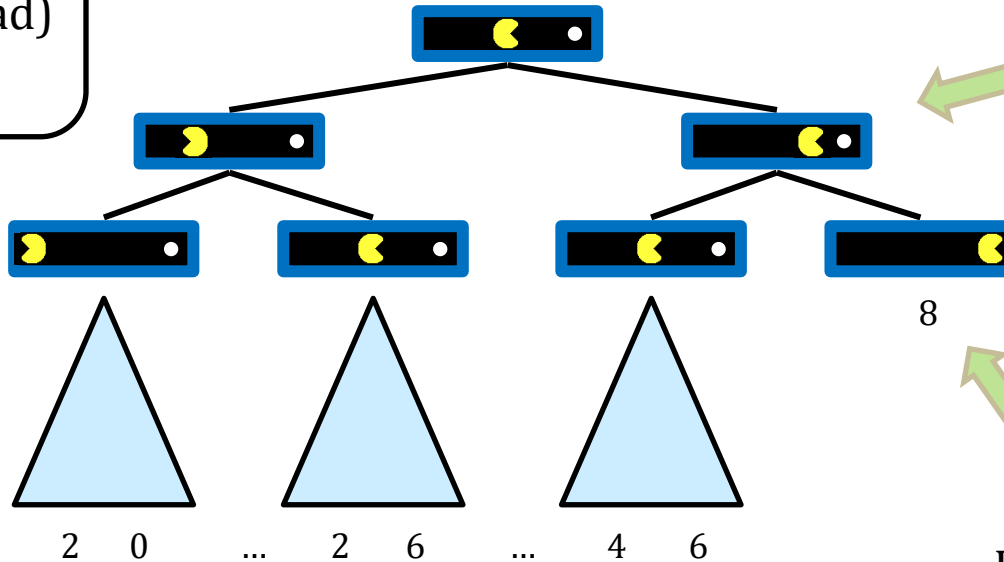
- ❑ Son problemas con características especiales
 - La accesibilidad de los estados depende de las acciones elegidas por el contrario
 - Dos tipos de soluciones diferentes (una para cada jugador)
 - No hay noción de optimalidad (todas las soluciones son iguales, no importa la longitud del camino)

Arboles de agentes únicos



Arboles de agentes únicos

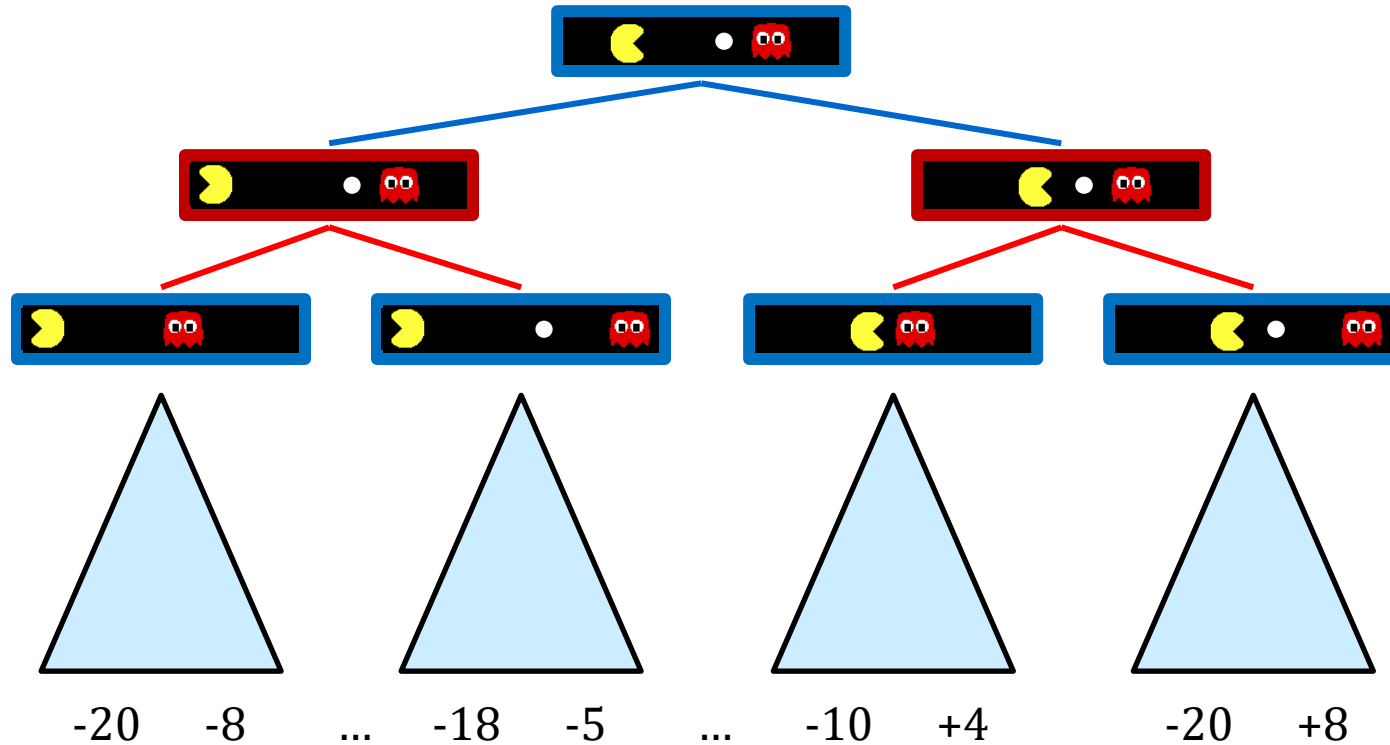
Valor de un estado: el
mejor resultado
alcanzable (utilidad)
de ese estado



Estado no terminal:
$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$

Estado terminal:
 $V(s) = \text{known}$

Árbol de juego de adversarios



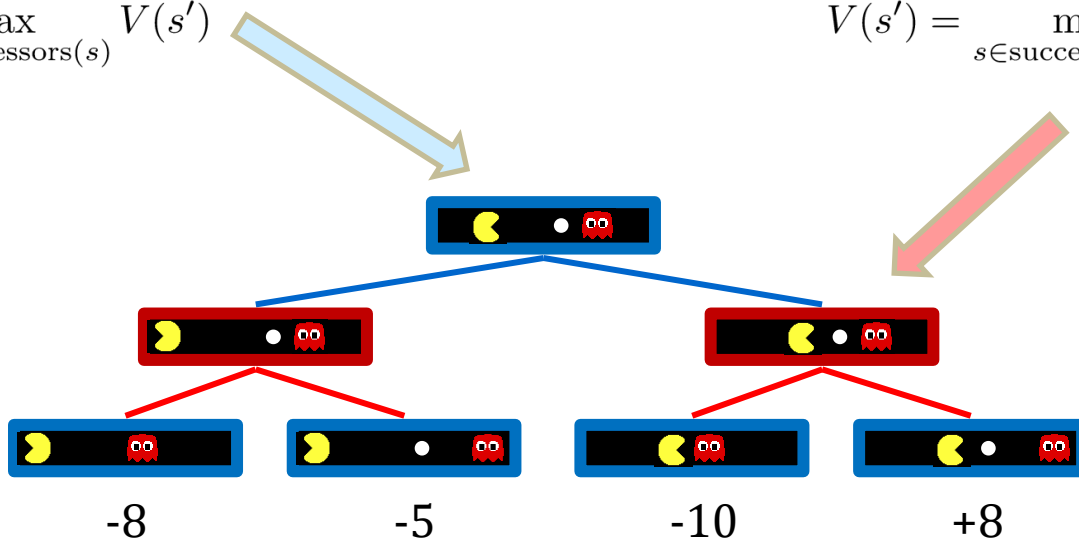
Árbol de juego de adversarios

Estado en control del agente:

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

Estado en control del oponente:

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$



Estados terminales:

$$V(s) = \text{known}$$

Árbol de juego de adversarios



MAX (X)



MIN (O)



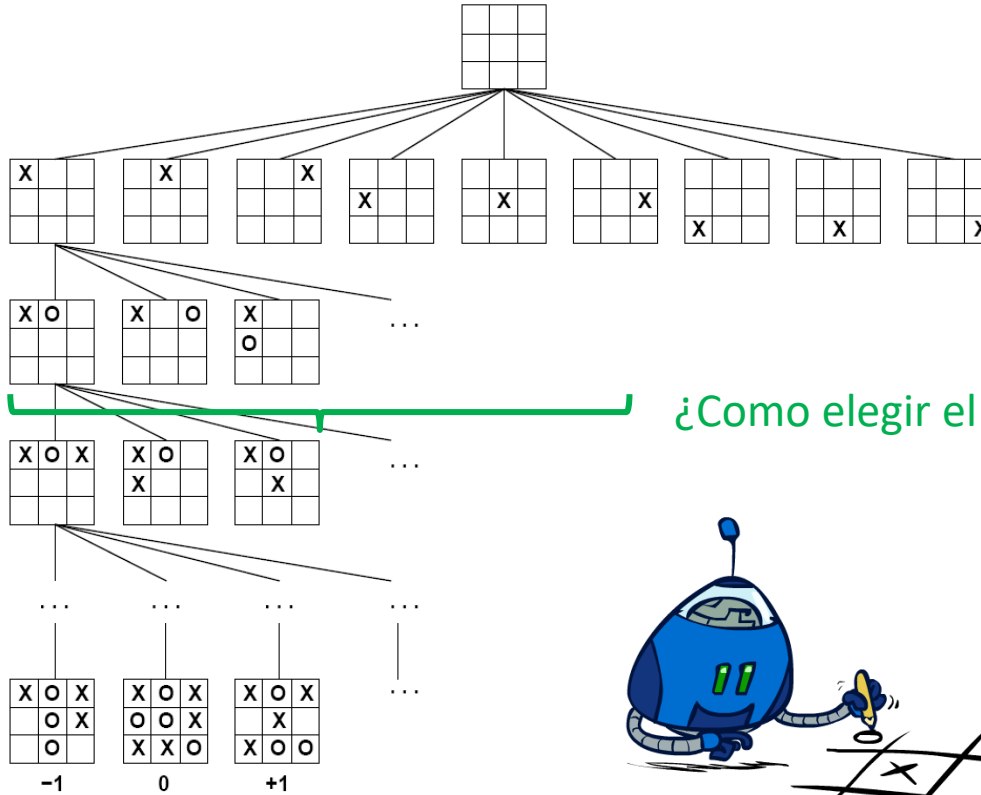
MAX (X)



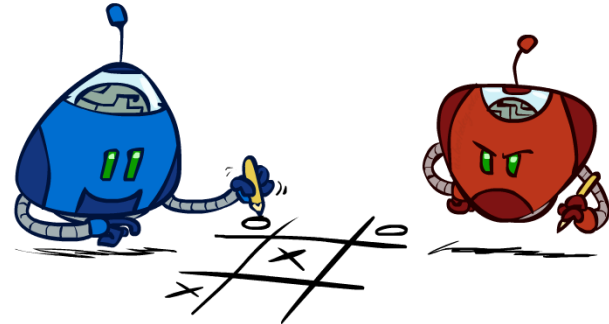
MIN (O)

TERMINAL

Utility



¿Como elegir el estado correcto?

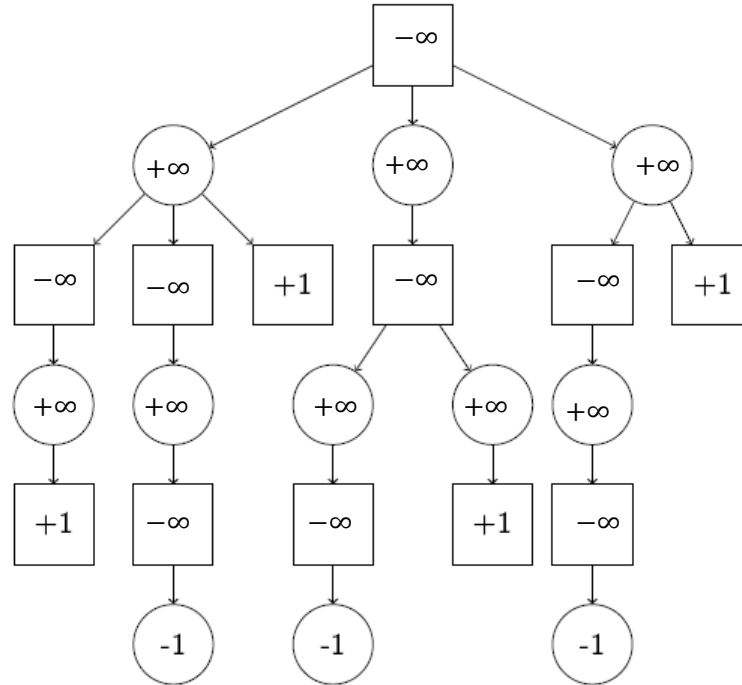




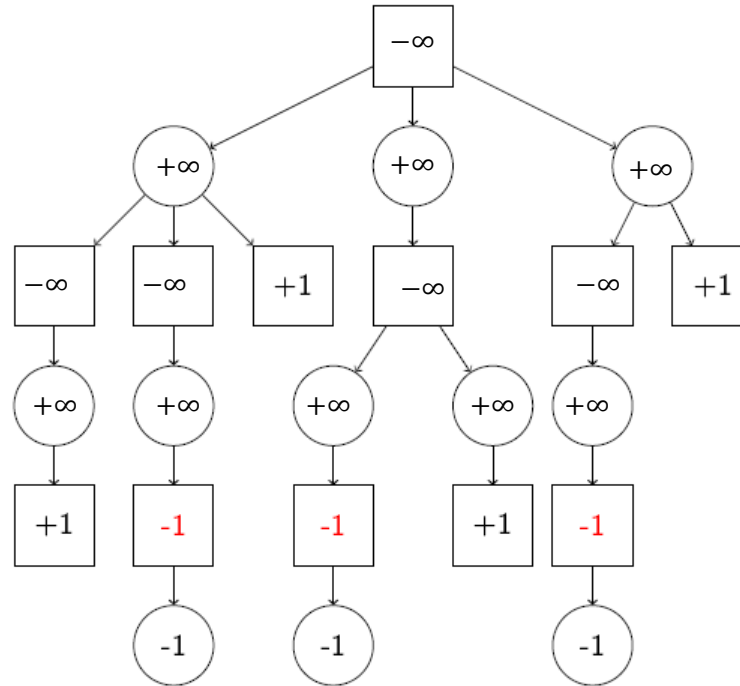
Búsqueda de adversarios

- ❑ La aproximación trivial es generar todo el árbol de jugadas
- ❑ Etiquetamos las jugadas terminales dependiendo de si gana (MAX = +1) o MIN (-1)
- ❑ El objetivo es encontrar un conjunto de movimientos accesible que de como ganador a MAX
- ❑ Se **propagan** los valores de las jugadas terminales de las hojas hasta la raíz, elegimos una rama de una hoja ganadora accesible
- ❑ Una búsqueda en profundidad minimiza el espacio
- ❑ En juegos mínimamente complejos esta búsqueda es **impracticable** (p.e.: ajedrez $O(2^{35})$, go $O(2^{300})$)

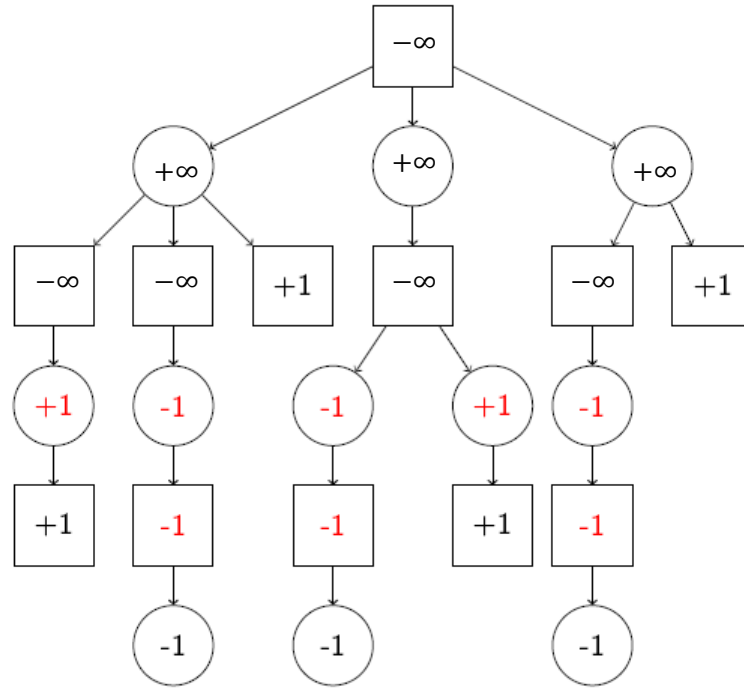
Búsqueda de adversarios



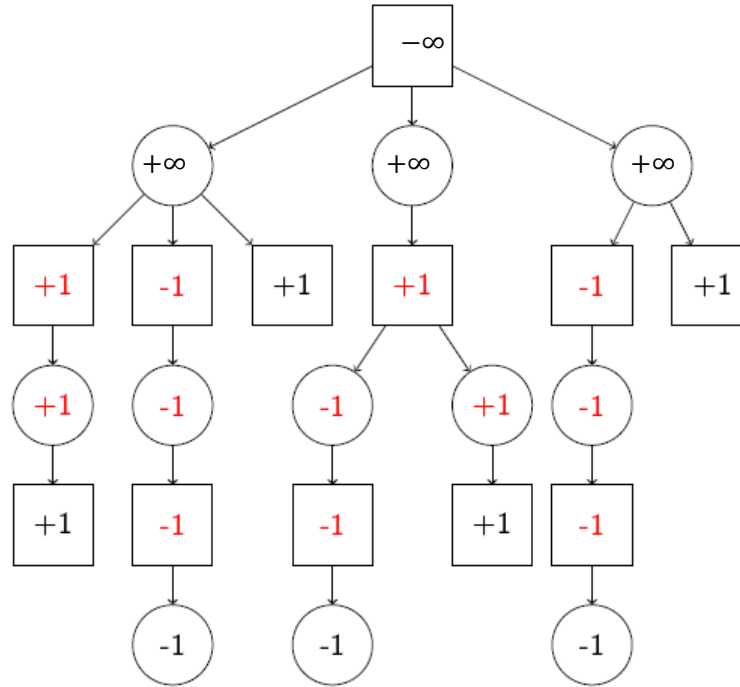
Búsqueda de adversarios



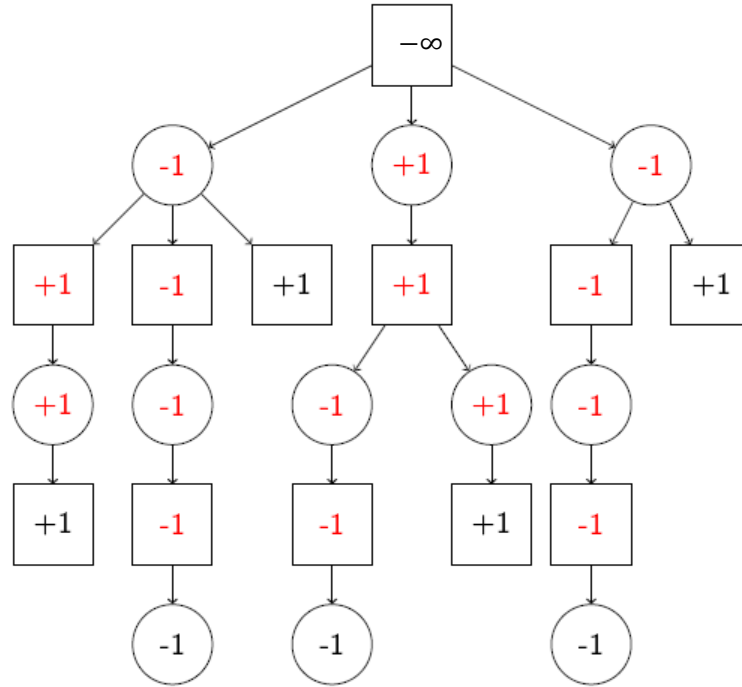
Búsqueda de adversarios



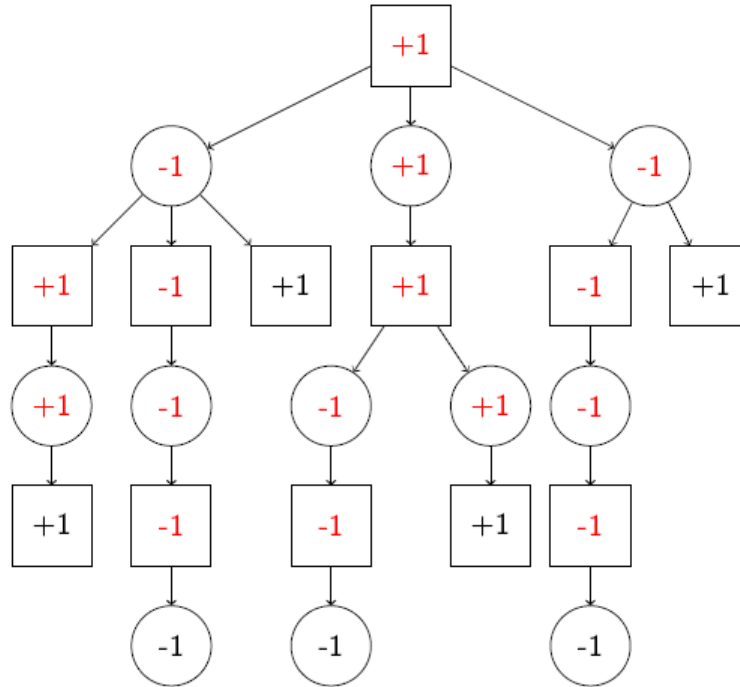
Búsqueda de adversarios



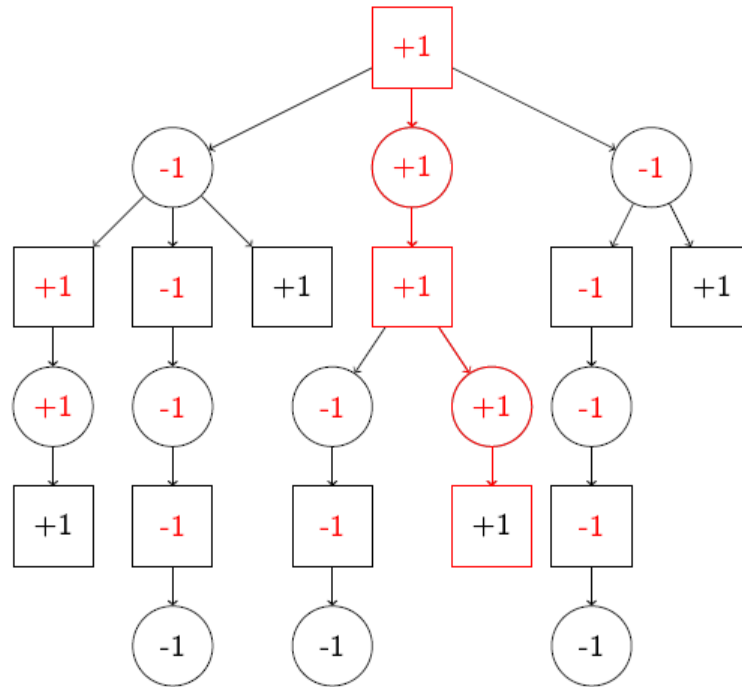
Búsqueda de adversarios



Búsqueda de adversarios



Búsqueda de adversarios





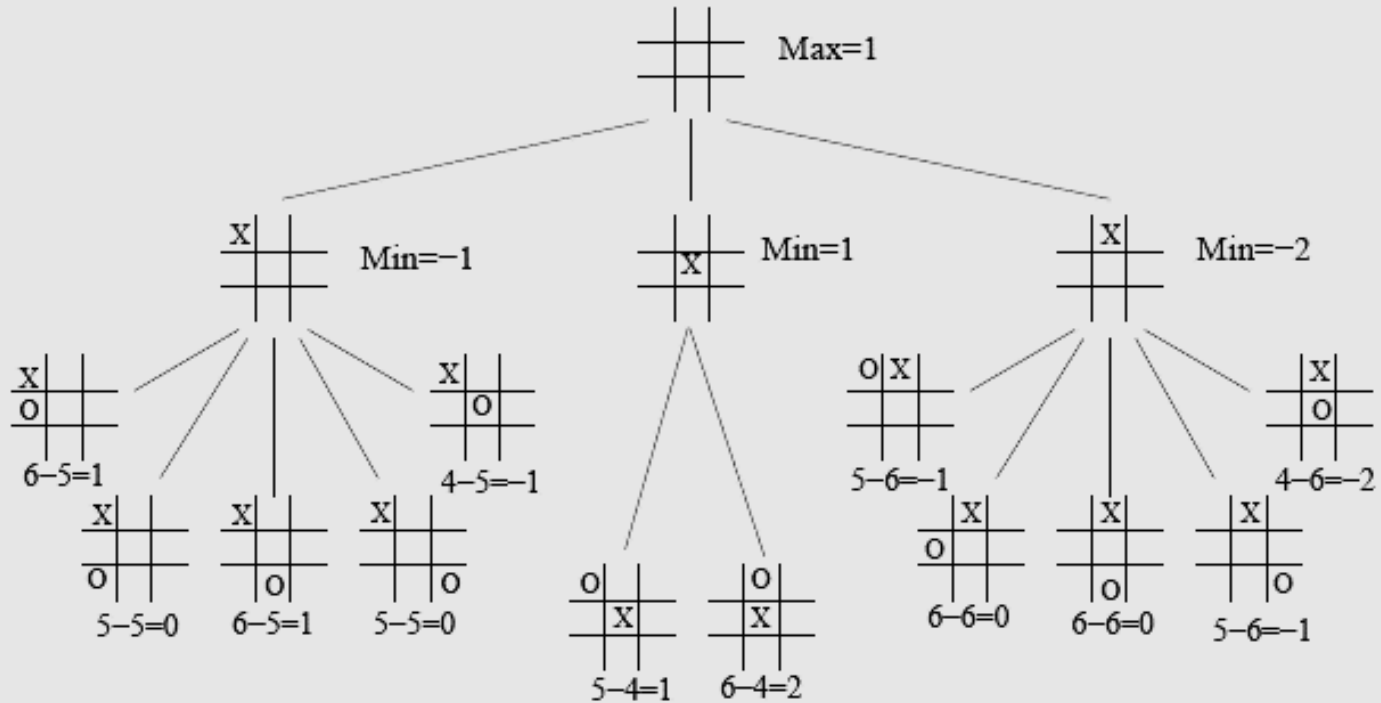
Búsqueda de adversarios

- ❑ **Aproximación heurística:** Definir una función que nos indique lo cerca que estamos de una jugada ganadora (o perdedora)
- ❑ En esta función intervendrá información del dominio
- ❑ Esta función **no representa ningún coste** ni es una distancia en pasos
- ❑ Por convención las jugadas ganadoras se evalúan a $+\infty$ y las perdedoras a $-\infty$
- ❑ El algoritmo busca con profundidad limitada y sólo decide la siguiente jugada a partir del nodo raíz
- ❑ Cada nueva decisión implicará **repetir la búsqueda**
- ❑ A mayor profundidad en la búsqueda mejor jugaremos

Juego: 3 en línea (michi)

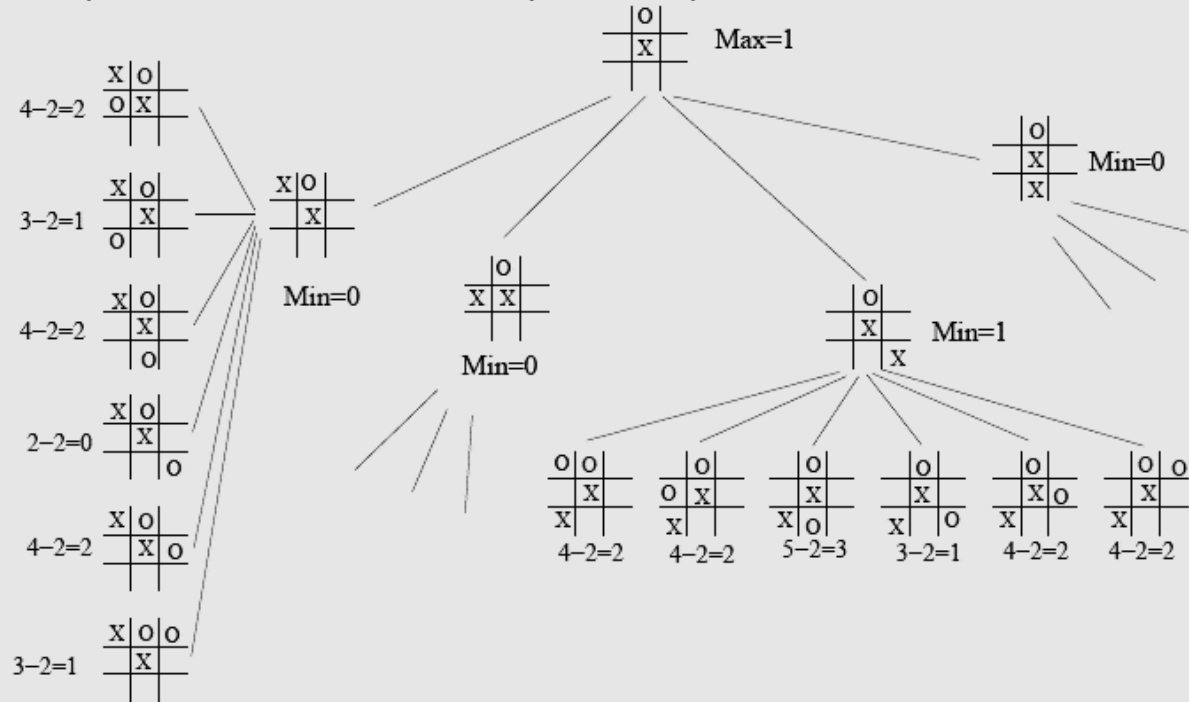
- ☐ e (función utilidad) = [número de filas, columnas y diagonales completas disponibles para MAX] – [número de filas, columnas y diagonales completas disponibles para MIN]
- ☐ MAX juega con X y desea maximizar e
- ☐ MIN juega con 0 y desea minimizar e
- ☐ Valores absolutos altos de e : buena posición para el que tiene que mover
- ☐ Controlar las simetrías
- ☐ Utilizar una profundidad de parada (en el ejemplo: 2)

Búsqueda de adversarios



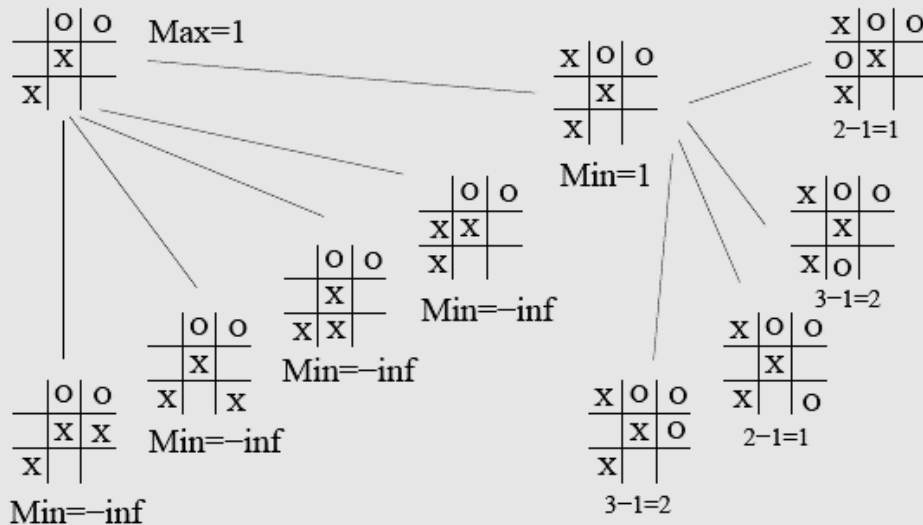
Búsqueda de adversarios

Suponiendo que MIN coloca su ficha en la posición superior de la columna central

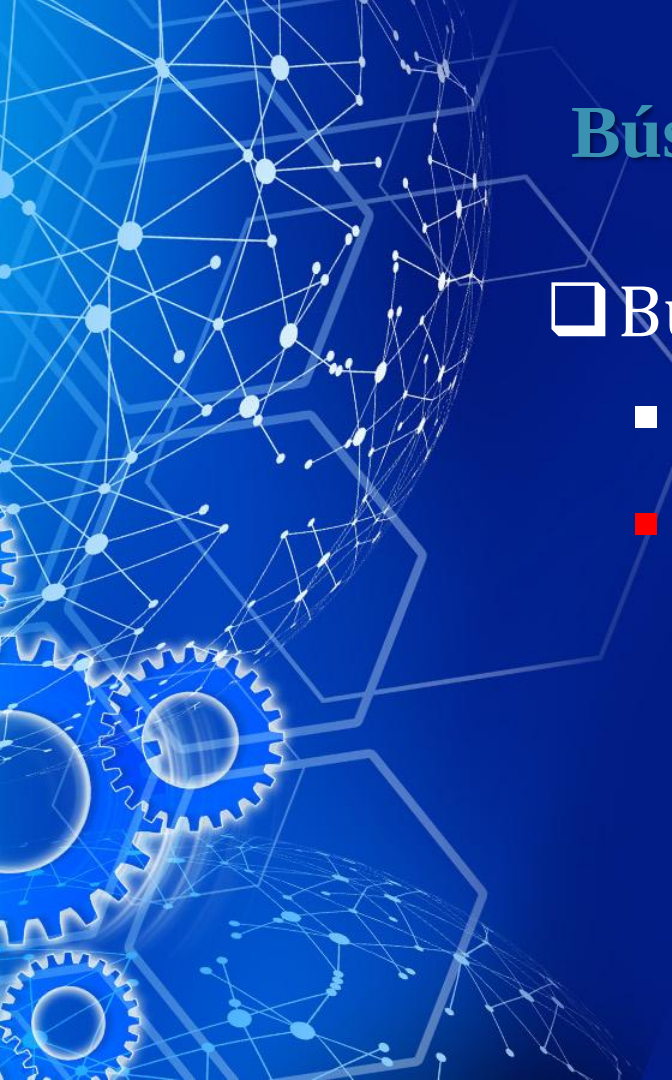


Búsqueda de adversarios

Suponiendo que MIN coloca su ficha en la esquina superior derecha



- ❑ Por convención:
 - las jugadas ganadoras se evalúan a “ $+\infty$ ”
 - las jugadas perdedoras se evalúan a “ $-\infty$ ”

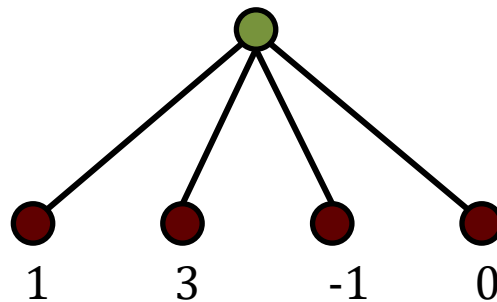


Búsqueda y Satisfacción de la Restricción

- Búsqueda entre adversarios
 - Juegos
 - **Algoritmo Minimax**

Algoritmo minimax

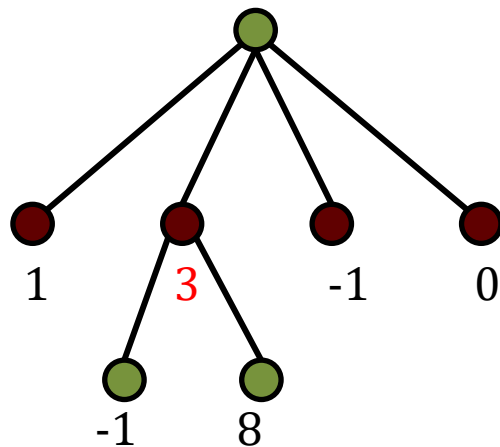
Considerando solo nuestros posibles movimientos...
... elegiríamos el movimiento mas prometedor (MAX):



Algoritmo minimax

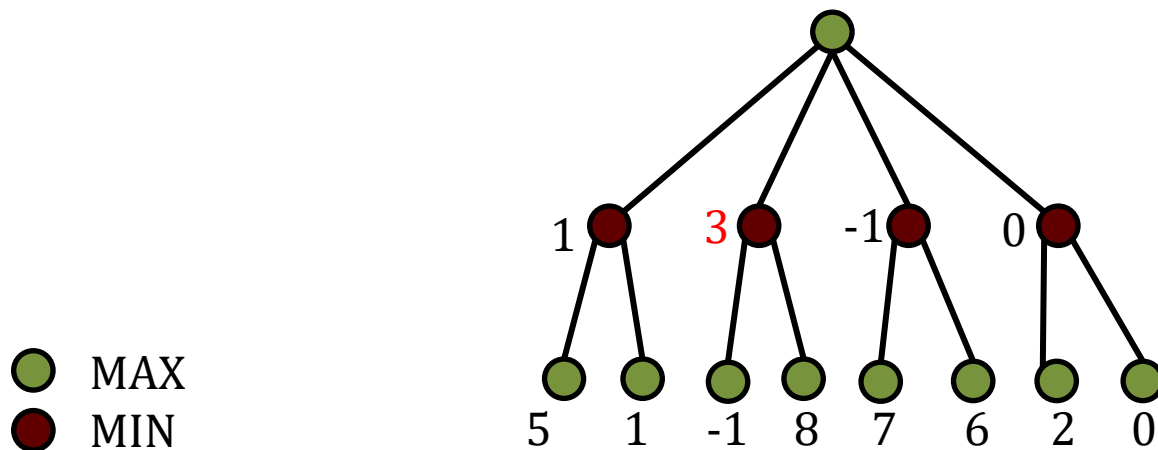
Nuestro oponente, a continuación...
... elegiría lo que mas le conviene a el (MIN):

● MAX
● MIN



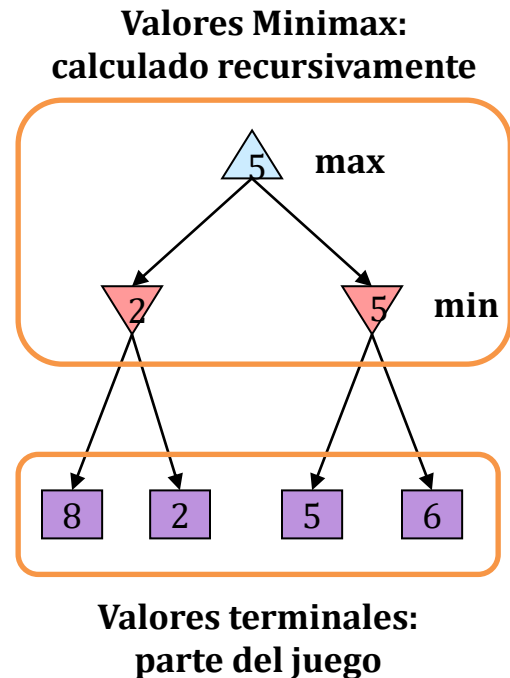
Algoritmo minimax

Si antes de realizar el movimiento, hubiésemos explorado un nivel mas de profundidad, nos habría dado cuenta de que el movimiento no era bueno:



Algoritmo minimax

- Juegos deterministas de suma cero:
 - Tres en línea, Ajedrez, Damas
 - Un jugador maximiza el resultado
 - El otro jugador minimiza resultado
- Búsqueda Minimax:
 - Un árbol de búsqueda en el espacio de estados
 - Los jugadores alternan los turnos
 - Calcula el valor **minimax** de cada nodo: la mejor utilidad alcanzable contra un adversario racional (optimo)



Algoritmo minimax

Implementación

def max-value(estado):

 inicializa $v = -\infty$

 Para cada successor del estado:

$v = \max(v, \text{min-value}(\text{successor}))$

 return v



def min-value(estado):

 inicializa $v = +\infty$

 Para cada successor del estado:

$v = \min(v, \text{max-value}(\text{successor}))$

 return v

$$V(s) = \max_{s' \in \text{successors}(s)} V(s')$$

$$V(s') = \min_{s \in \text{successors}(s')} V(s)$$

Algoritmo minimax

def value(estado):

si el estado es un estado terminal: retorna la utilidad del estado

si el siguiente agente es MAX: retorna max-value(estado)

si el siguiente agente es MIN: retorna min-value(estado)

def max-value(state):

inicializa $v = -\infty$

Para cada successor del estado:

$v = \max(v, \text{value}(\text{successor}))$

retorna v

def min-value(state):

inicializa $v = +\infty$

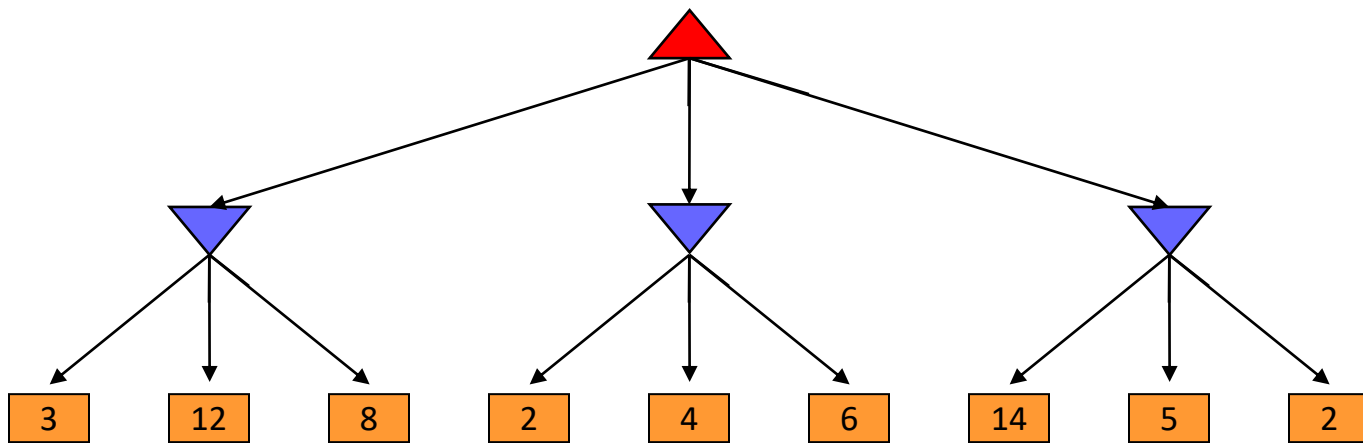
Para cada successor del estado:

$v = \min(v, \text{value}(\text{successor}))$

retorna v

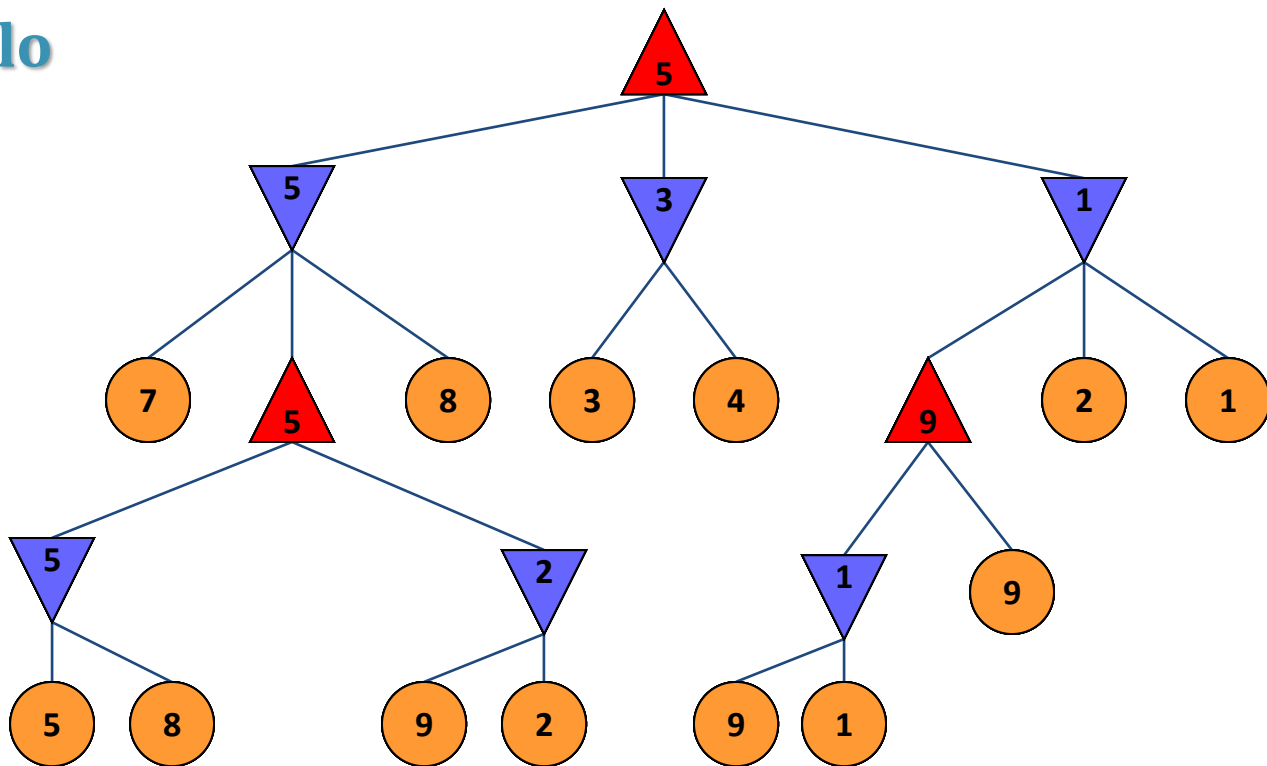
Algoritmo minimax

Ejemplo



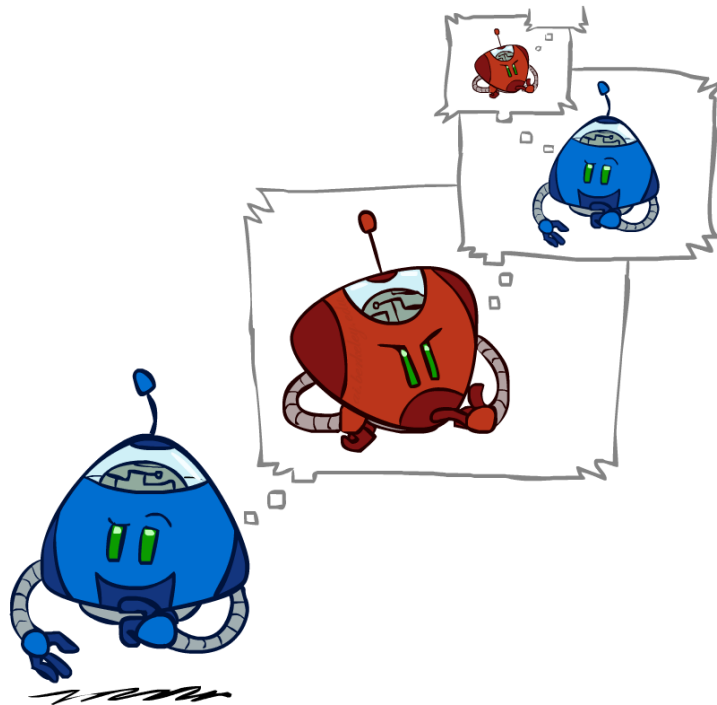
Algoritmo minimax

Ejemplo



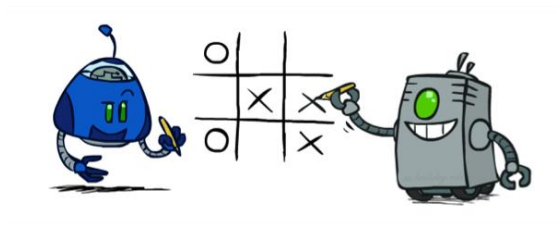
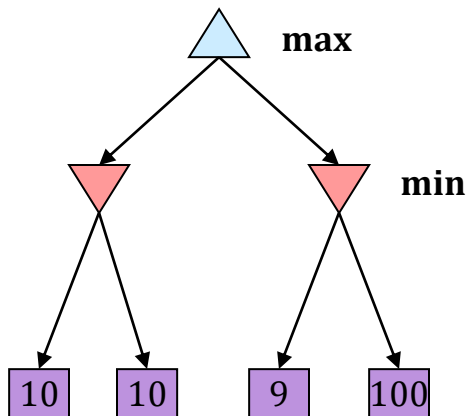
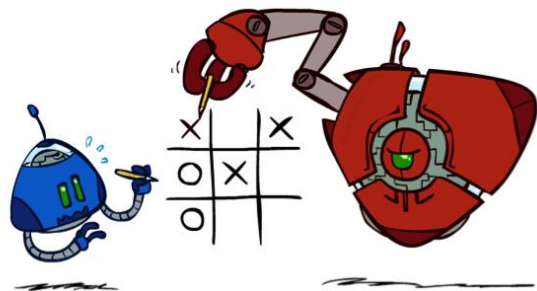
Eficiencia

- ¿Que tan eficiente es minimax?
 - Igual al DFS (exhaustiva)
 - Tiempo: $O(b^m)$
 - Espacio: $O(bm)$
- Ejemplo: Para ajedrez, $b \approx 35$, $m \approx 100$
 - La solución exacta es completamente inviable
 - Pero, ¿Necesitamos explorar todo el árbol?



Algoritmo minimax

Propiedades

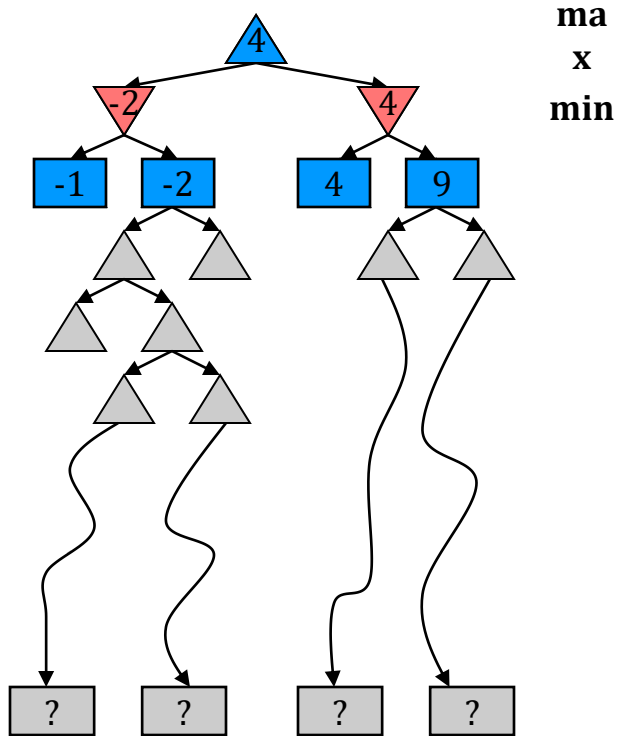


Optimo contra un jugador perfecto. ¿De otra forma?

Algoritmo minimax

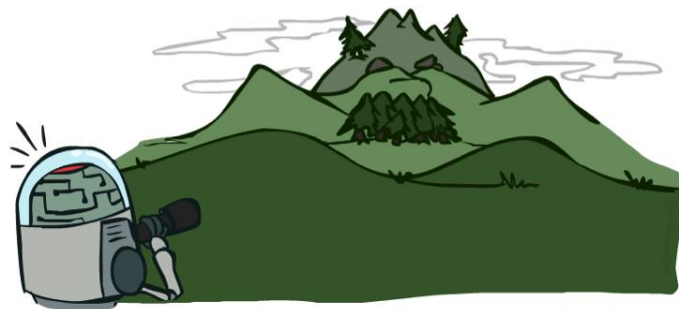
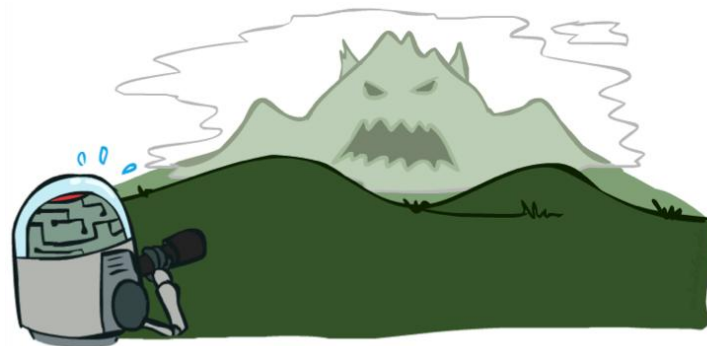
Limite de los recursos

- Problema: En juegos reales, no puedes buscar todas las hojas!
- Solución: Búsqueda de profundidad limitada
 - Buscar hasta una profundidad limitada en el árbol
 - Remplace la utilidad de las terminaciones con una función de evaluación para las posiciones de estas terminales
- Ejemplo:
 - Suponer que se tiene 100 segundos, y puede explorar 10K nodos / sec
 - Entonces puede checar 1M nodos por movimiento
 - α - β alcanza una profundidad de 8 – programa de ajedrez decente
- La garantía de un juego optimo ya no esta
- Mayor numero de capas hace un GRAN diferencia
- Utilizar la profundización iterativa



La profundidad importa

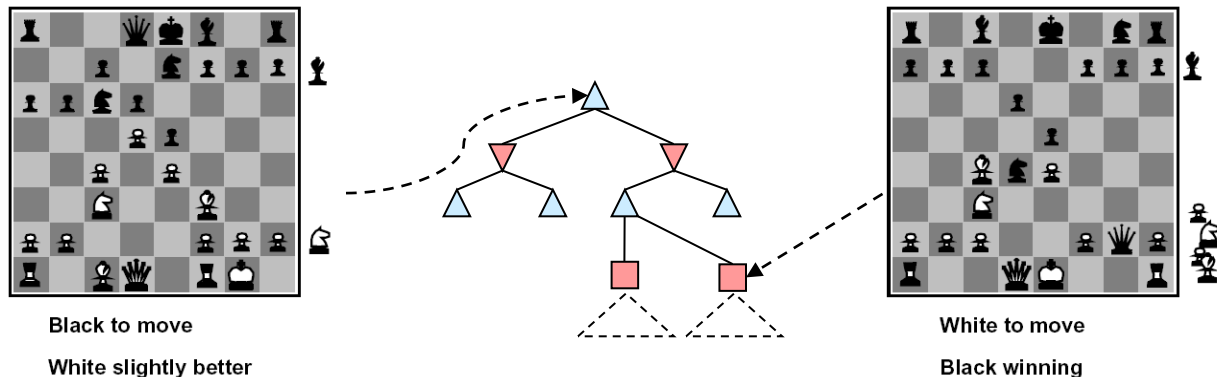
- ❑ Las funciones de evaluación son siempre imperfectas
- ❑ Mientras más profunda se encuentre la función de evaluación en el árbol, menos importa la calidad de esta función
- ❑ Esto es un importante ejemplo de la compensación entre la complejidad de las características y la complejidad de cálculo



Algoritmo minimax

Funciones de evaluación

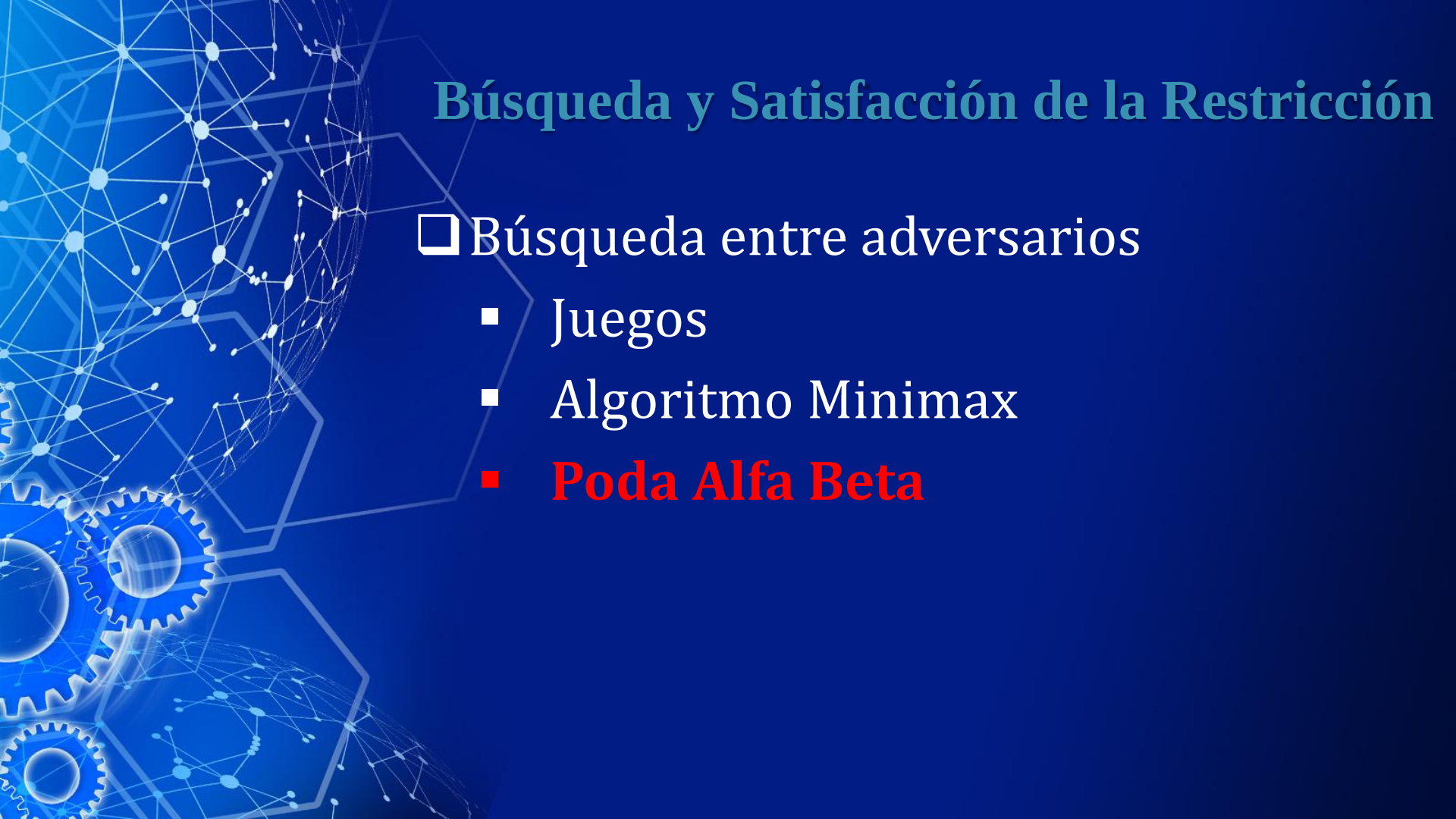
- ❑ Las funciones de evaluación puntúan las terminales en la búsqueda de profundidad limitada



- ❑ Función Ideal: retorna el valor minimax real de la posición
- ❑ En la practica: Es una suma lineal ponderada de las características:

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

- e.g. $f_1(s) = (\text{numero de reinas blancas} - \text{numero de reinas negras})$, etc.



Búsqueda y Satisfacción de la Restricción

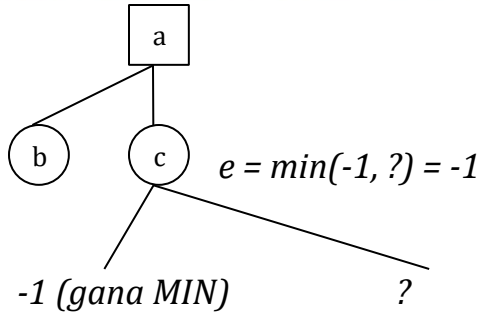
- Búsqueda entre adversarios
 - Juegos
 - Algoritmo Minimax
 - **Poda Alfa Beta**



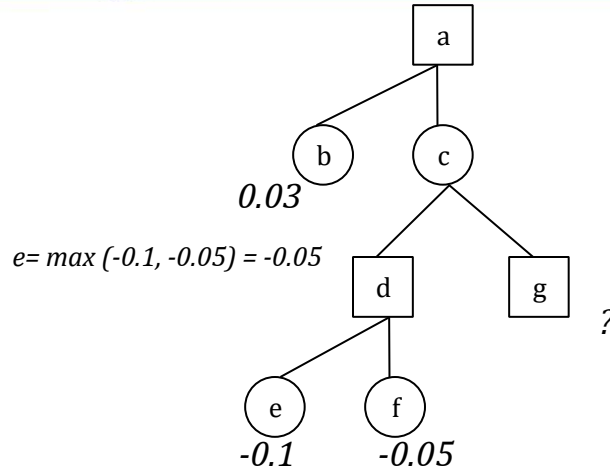
Poda alfa-beta

- ❑ Problema de la búsqueda minimax: el número de estados que tiene que examinar es exponencial con el número de movimientos.
- ❑ El exponente no se puede eliminar, pero se puede dividir en la mitad.
- ❑ Es posible calcular la decisión minimax correcta sin mirar todos los nodos en el árbol.
- ❑ La **poda alfa-beta** permite eliminar partes grandes del árbol, sin influir en la decisión final.

Minimax con poda α - β



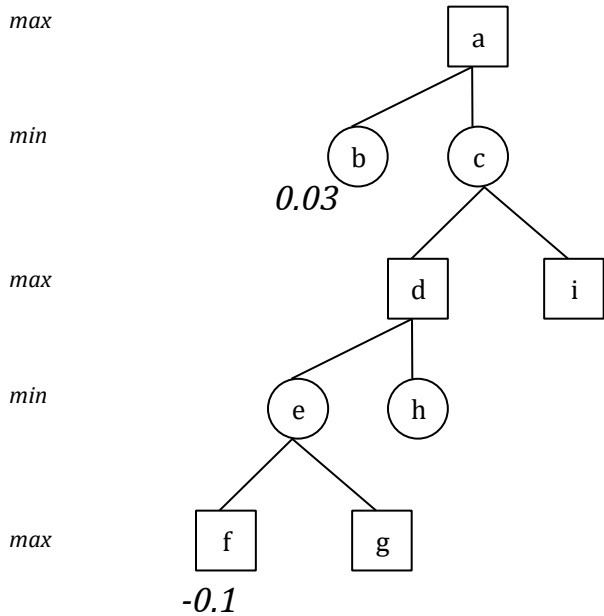
No tiene sentido seguir buscando los otros descendientes de c.



*En c: $e = \min(-0.05, v(g))$ por lo tanto en a:
 $e = \max(0.03, \min(-0.05, v(g))) = 0.03$
Se pueden pues podar los nodos bajo g; no aportan nada.*

El valor de la raíz y la decisión minimax son independientes de los valores de las hojas podadas.

Minimax con poda α - β



$$e(e) = \min(-0.1, v(g))$$

Como la rama b ya da un 0.03 ,

Cualquier cosa peor no sirve

=> No hay que explorar g

$$e(d) = \max(e(e), h)$$

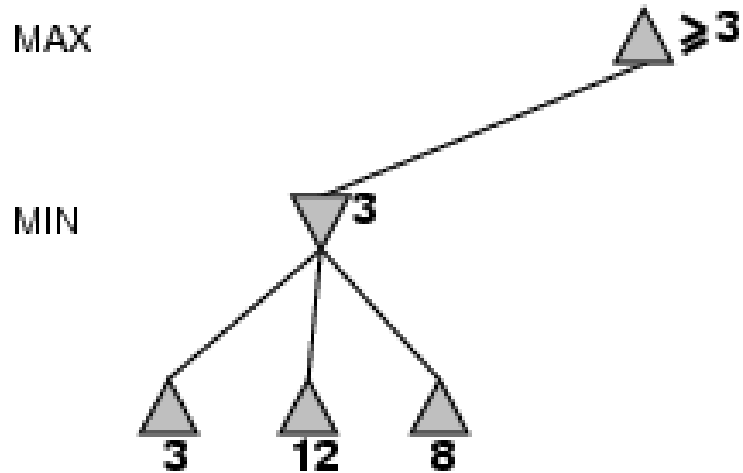
=> Sí hay que explorar h

...

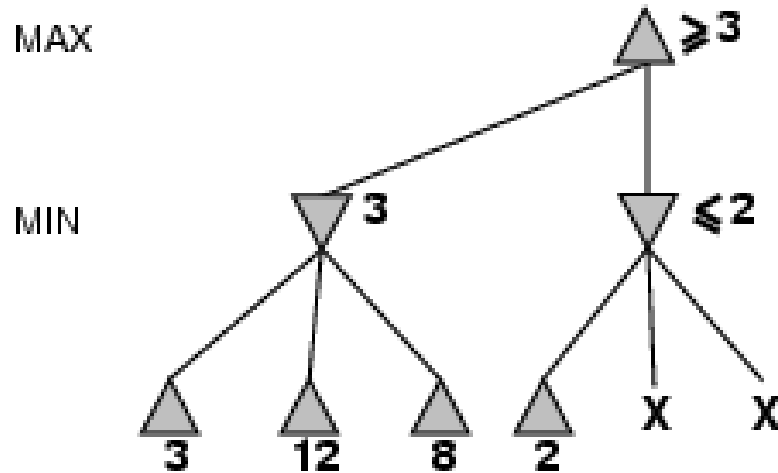
La búsqueda minimax es primero en profundidad: en cualquier momento **sólo se consideran los nodos a lo largo de un camino del árbol.**

- ❑ Los dos parámetros alfa y beta describen los límites sobre los valores que aparecen a lo largo del camino:
 - α = el valor de la mejor opción (el más alto) que se ha encontrado hasta el momento en cualquier punto del camino, para MAX
 - β = el valor de la mejor opción (el más bajo) que se ha encontrado hasta el momento en cualquier punto del camino, para MIN
- ❑ La búsqueda alfa-beta actualiza el valor de α y β según se va recorriendo el árbol y termina la recursión cuando encuentra un nodo peor que el actual valor α o β correspondiente.

Ejemplo

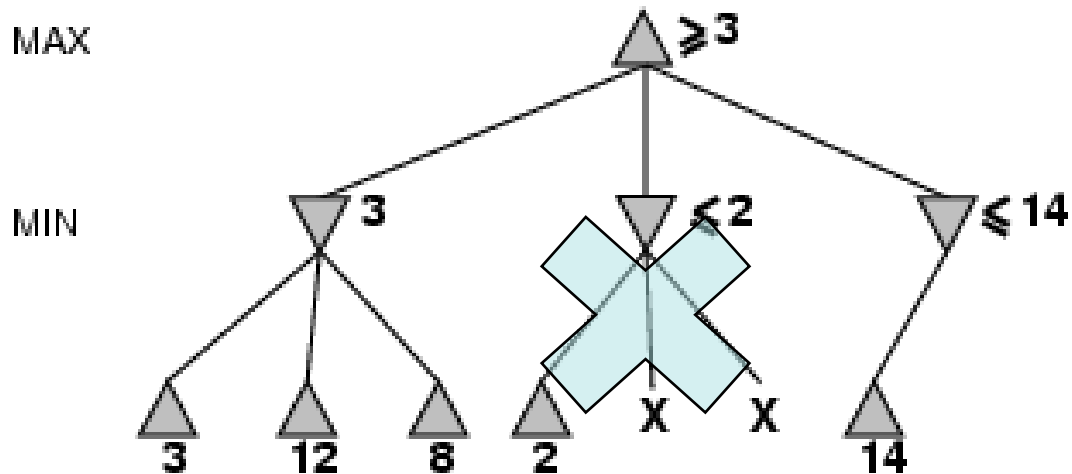


Ejemplo



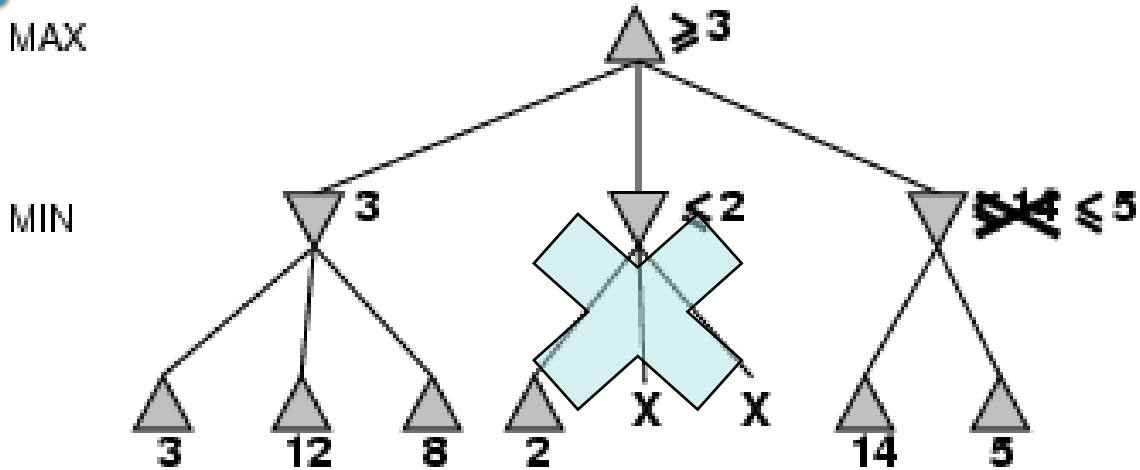
Poda alfa-beta

Ejemplo

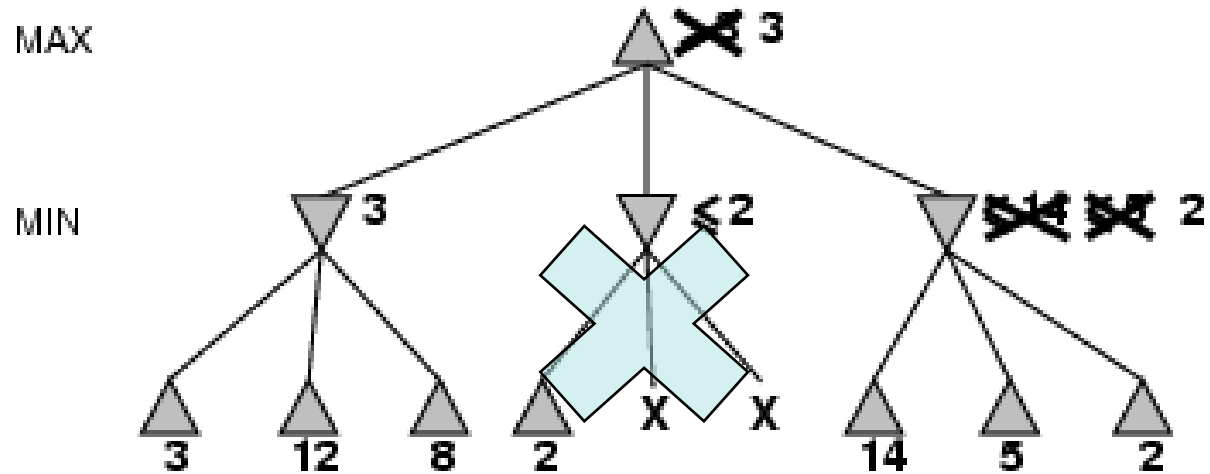


Poda alfa-beta

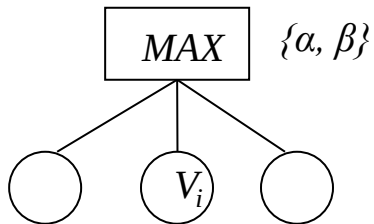
Ejemplo



Ejemplo

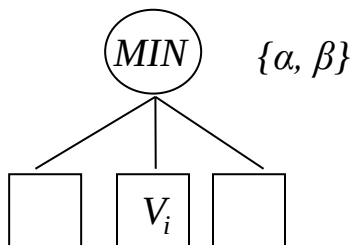


Poda alfa-beta



*Si $V_i \geq \beta$ poda β
Si $V_i > \alpha$ modificar α*

Retornar α



*Si $V_i \leq \alpha$ poda α
Si $V_i < \beta$ modificar β*

Retornar β

Las cotas α y β se transmiten de padres a hijos de 1 en 1 y en el orden de visita de los nodos.

Algoritmo Minimax con poda α - β

```
Funcion valorMax (g,alpha,beta) retorna entero
Si estado_terminal(g) entonces
    retorna(evaluacion(g))
si no
    Para cada mov en movs_posibles(g)
        alfa=max(alfa , valorMin( aplicar(mov,g) , alfa , beta))
    si alfa >= beta entonces retorna(beta)
    fPara
        retorna(alfa)
    fsi
fFuncion
```

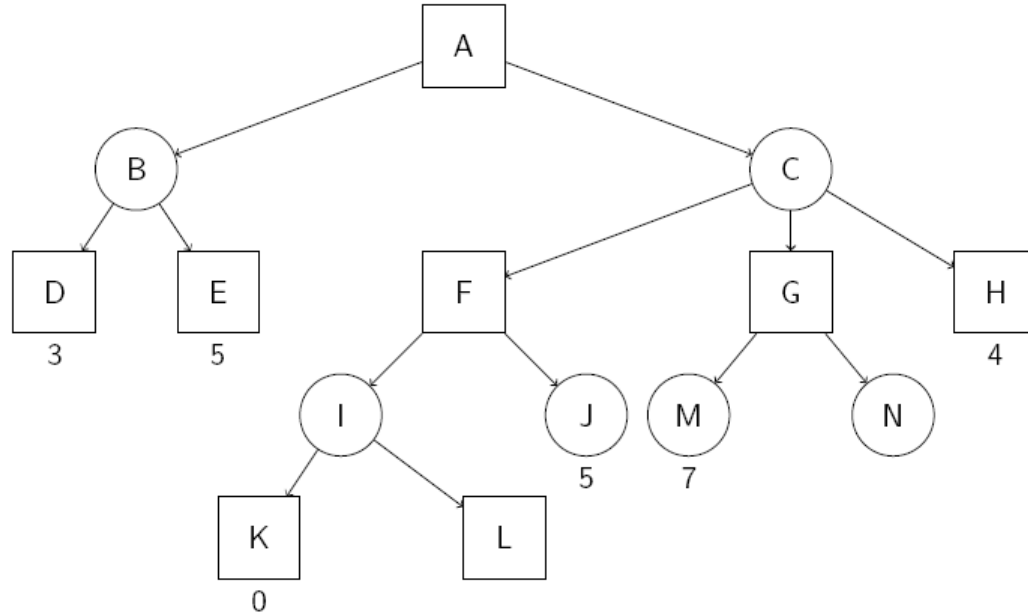
```
Funcion valorMin (g,alpha,beta) retorna entero
Si estado_terminal(g) entonces
    retorna(evaluacion(g))
si no
    Para cada mov en movs_posibles(g)
        beta=min(beta , valorMax( aplicar(mov,g) , alfa , beta))
    si alfa >= beta entonces retorna(alfa)
    fPara
        retorna(beta)
    fsi
fFuncion
```

El recorrido se inicia llamando a la función valorMax con $\alpha=-\infty$ y $\beta=+\infty$.

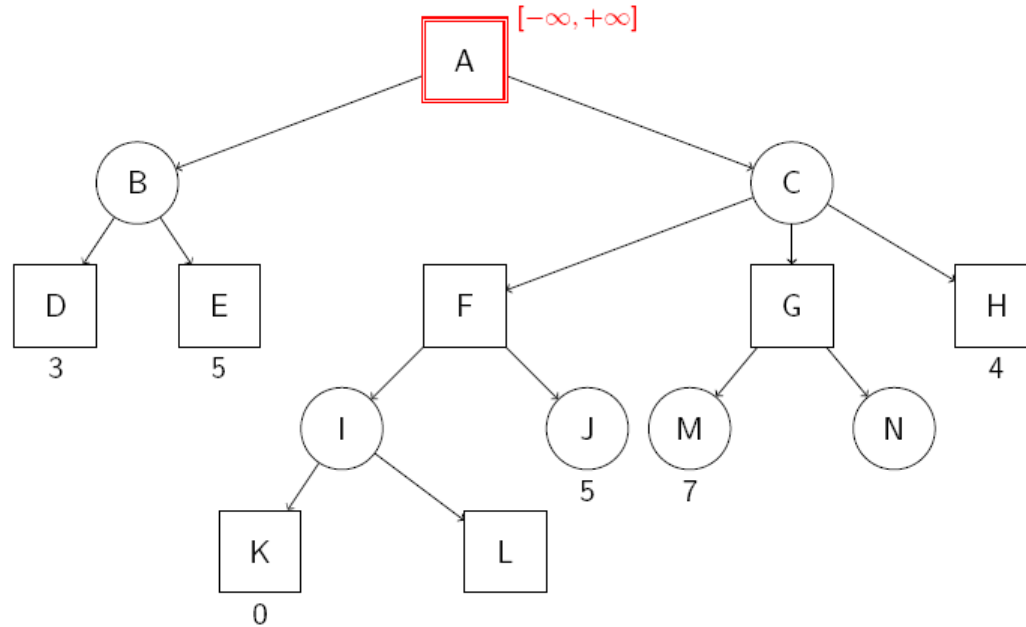
En la función valorMax α es el valor que se actualiza.

En la función valorMin β es el valor que se actualiza.

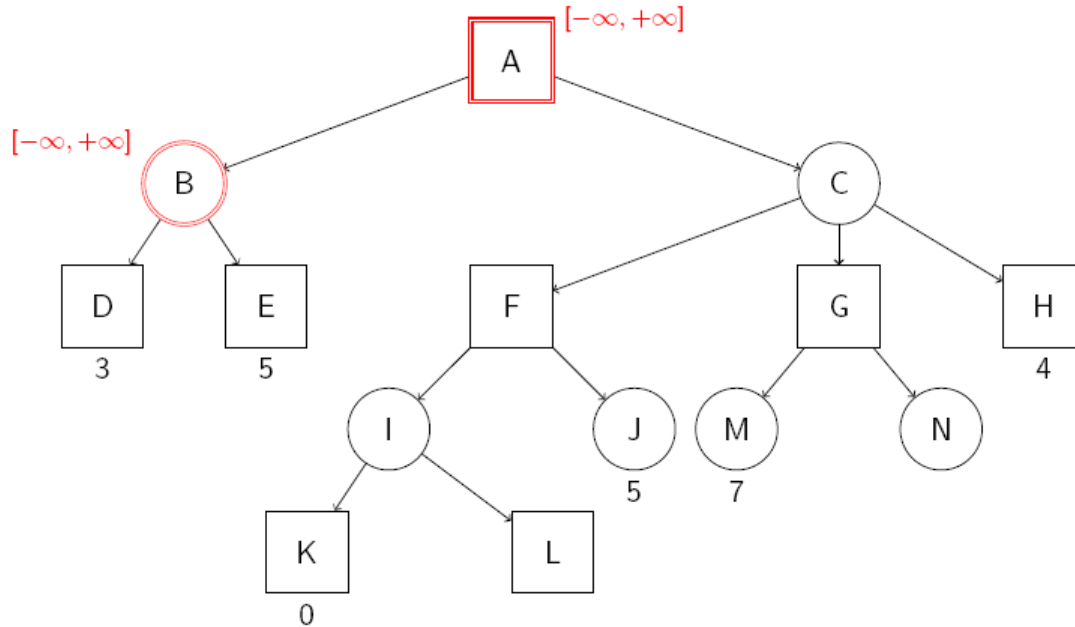
Ejemplo



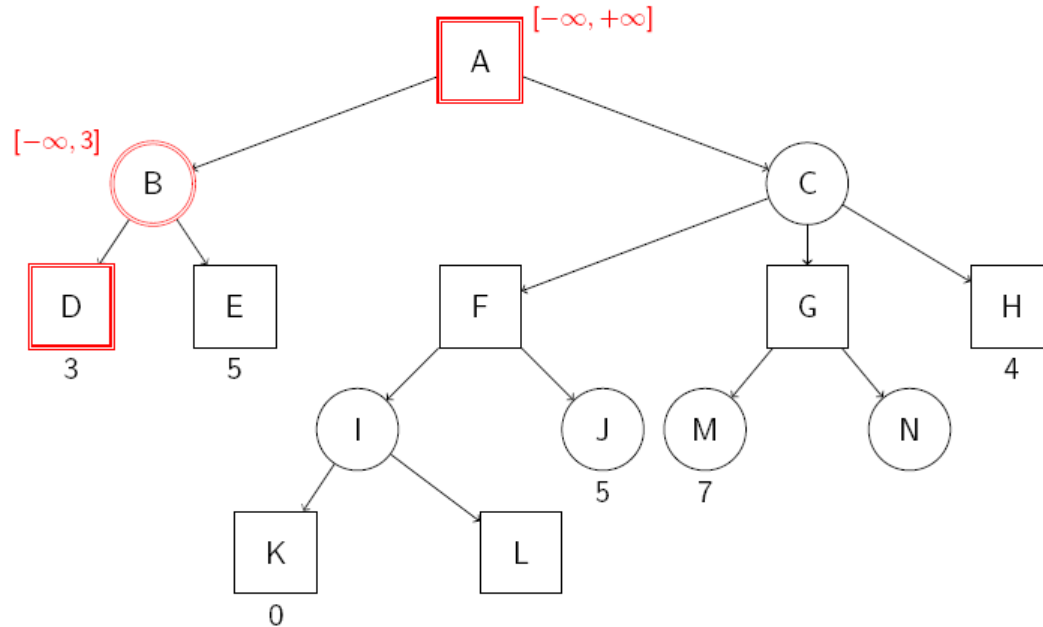
Ejemplo



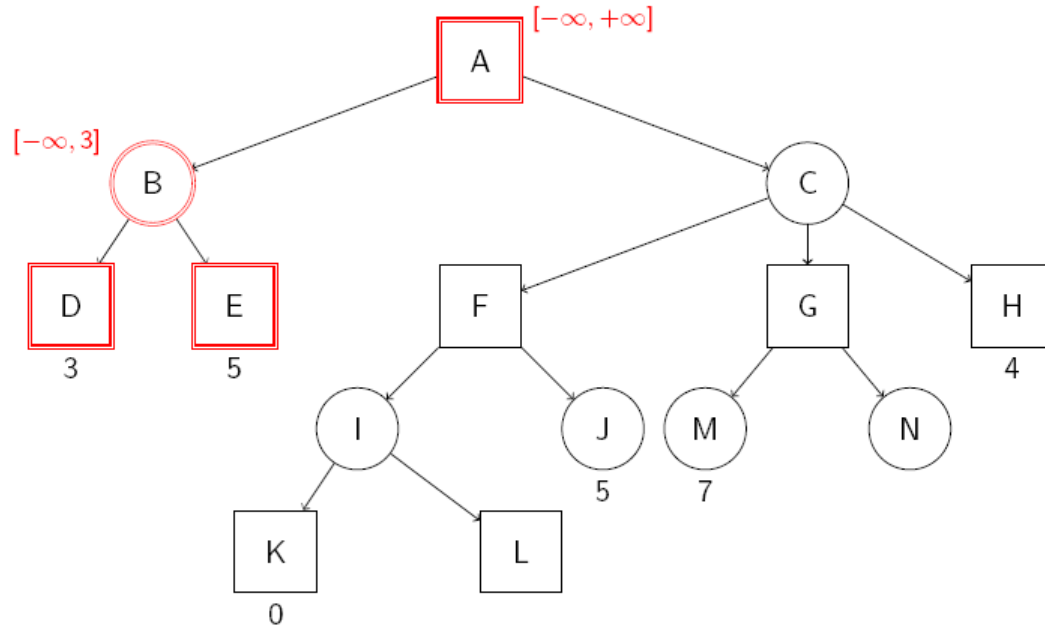
Ejemplo



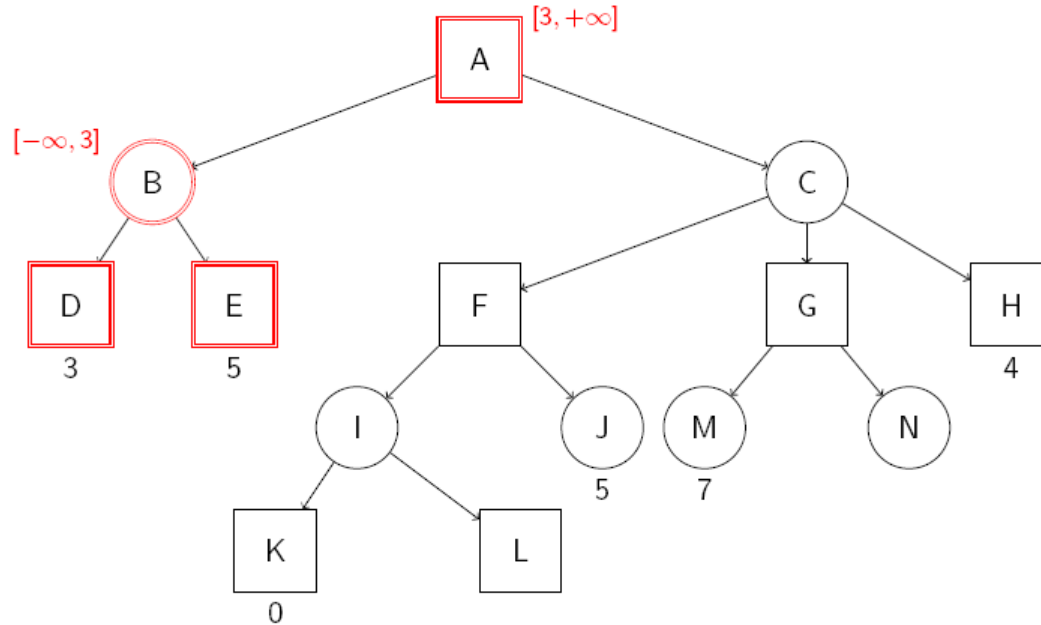
Ejemplo



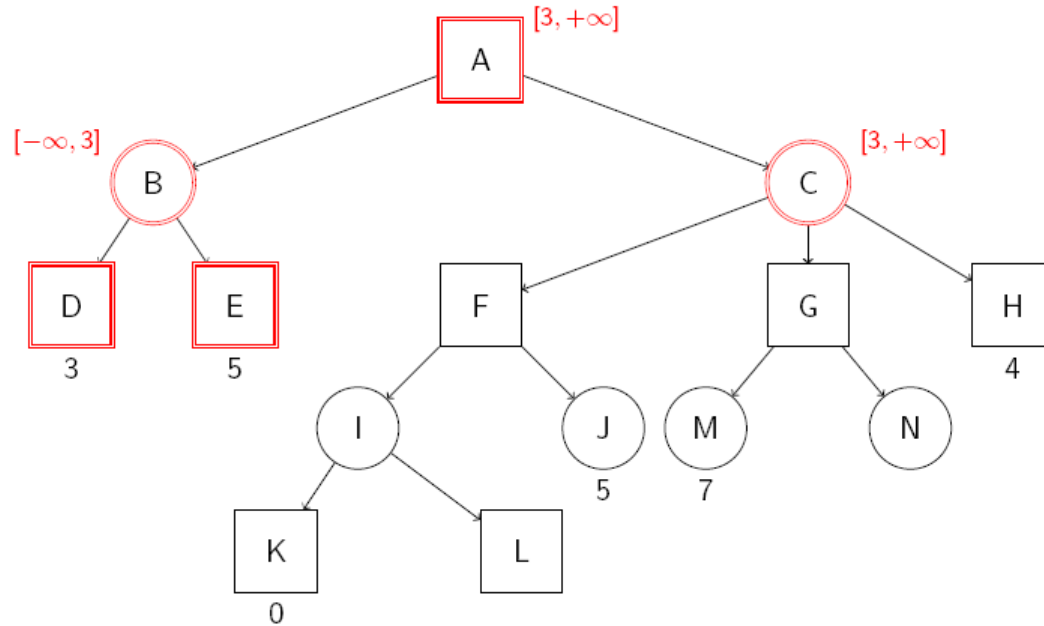
Ejemplo



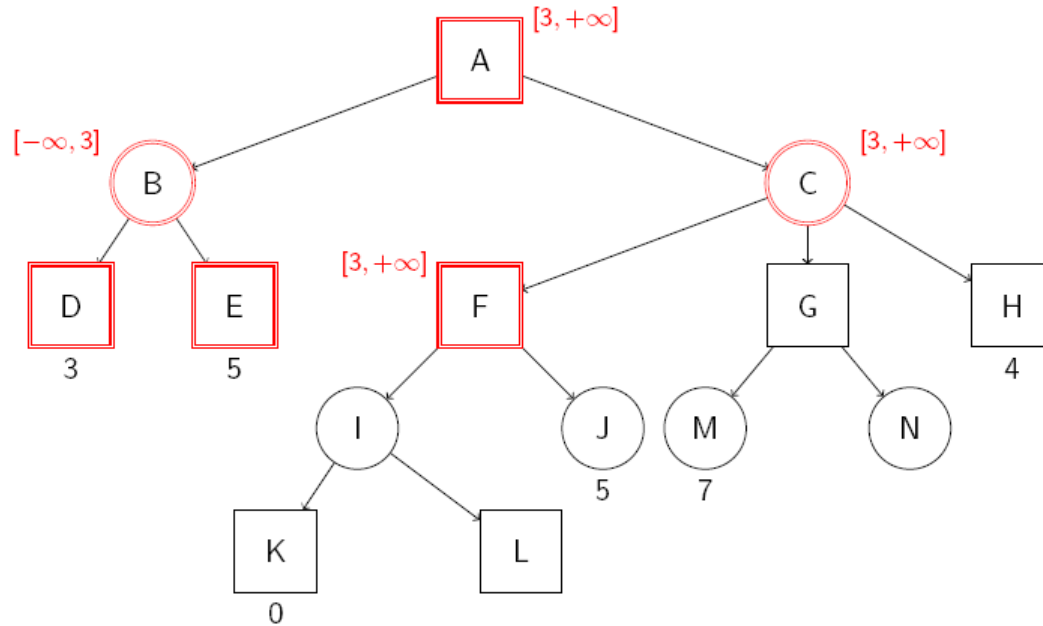
Ejemplo



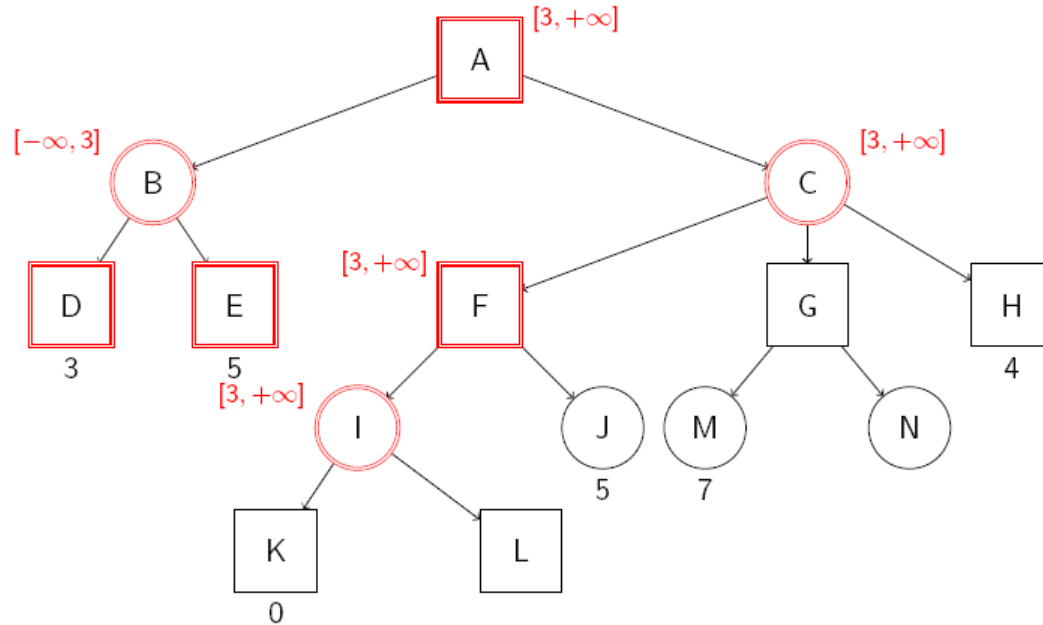
Ejemplo



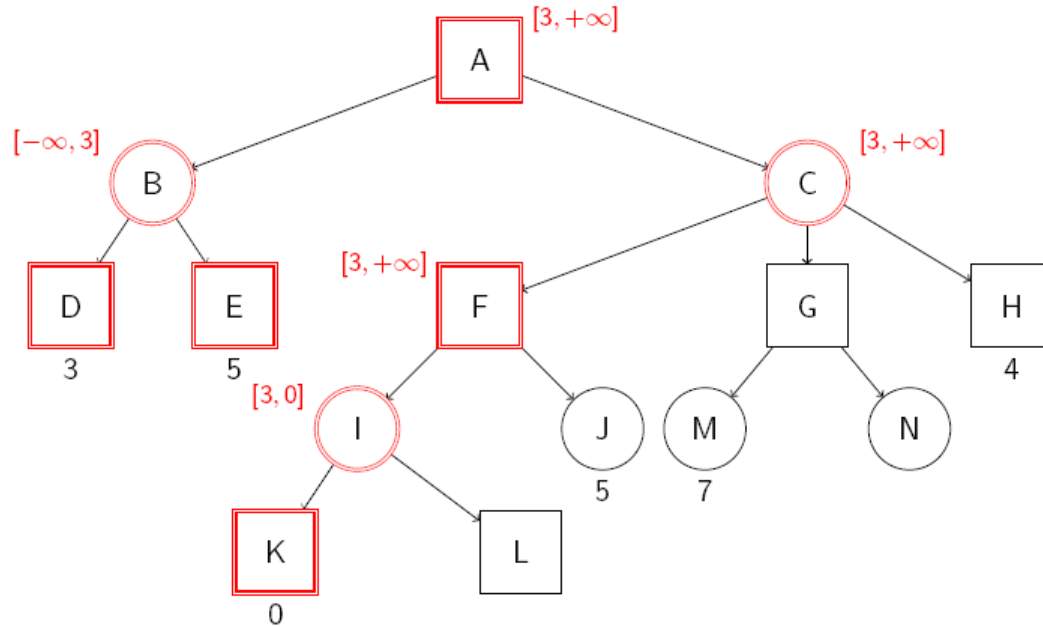
Ejemplo



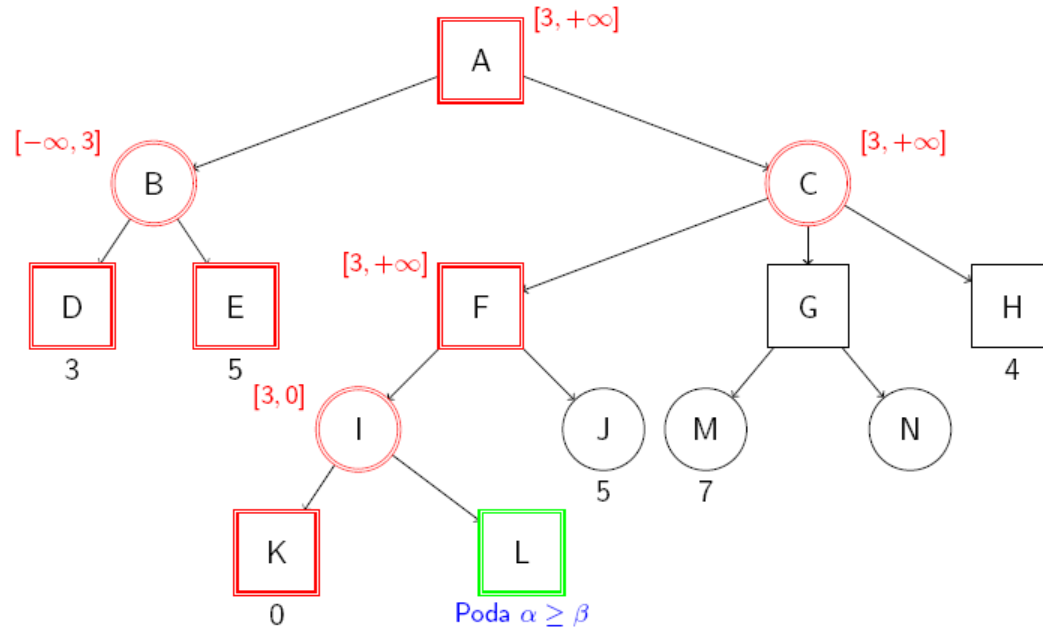
Ejemplo



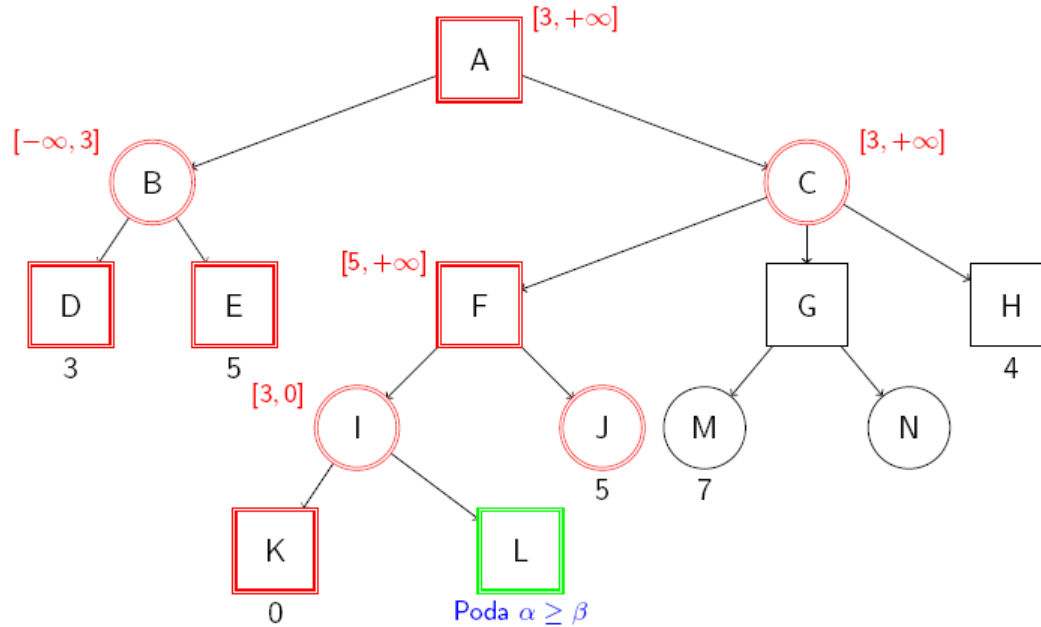
Ejemplo



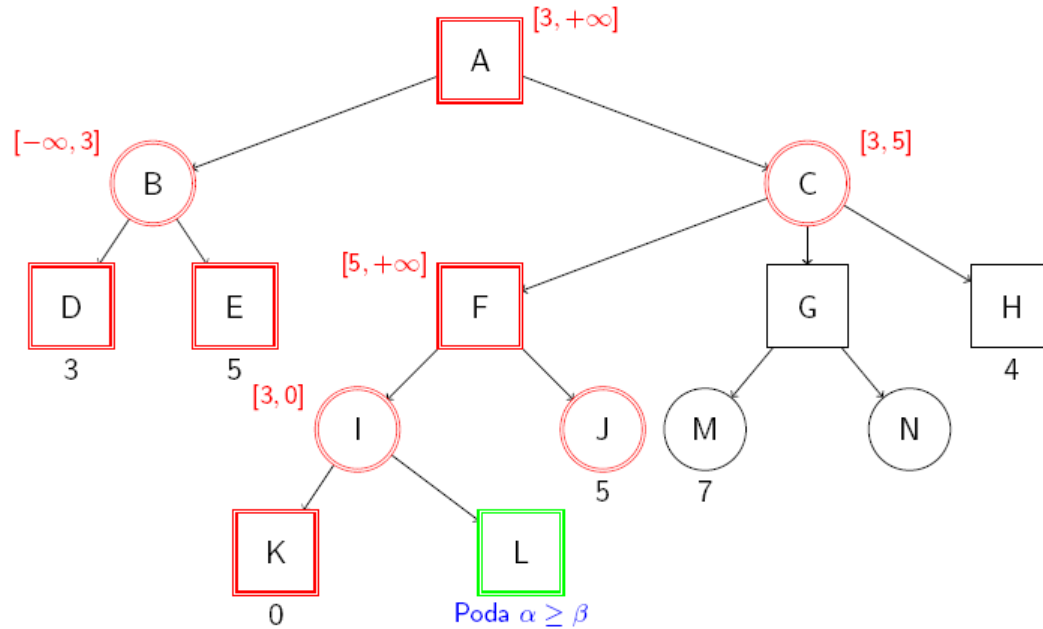
Ejemplo



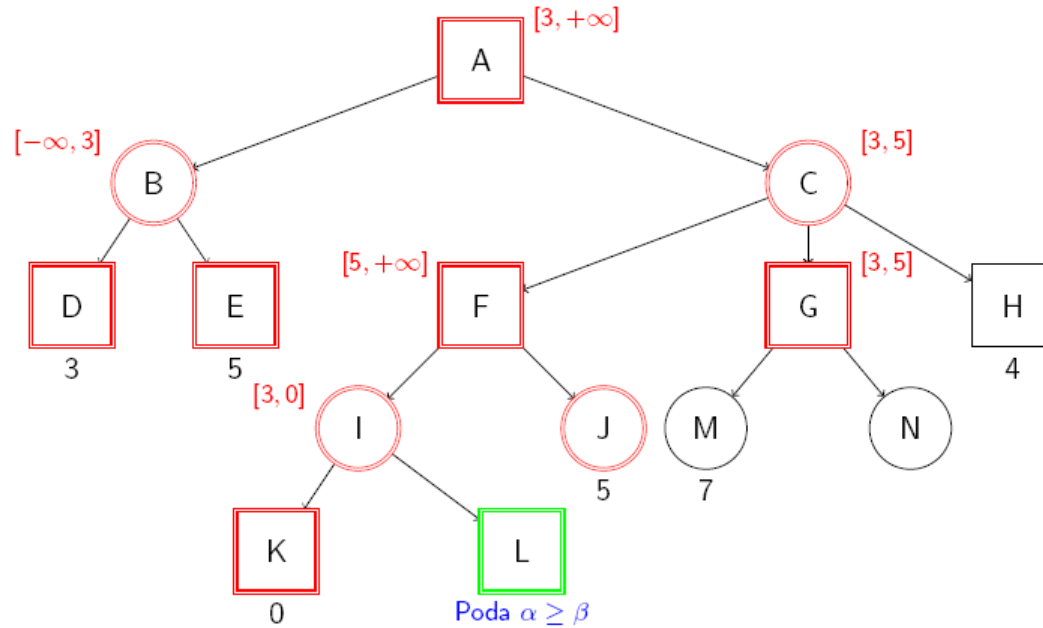
Ejemplo



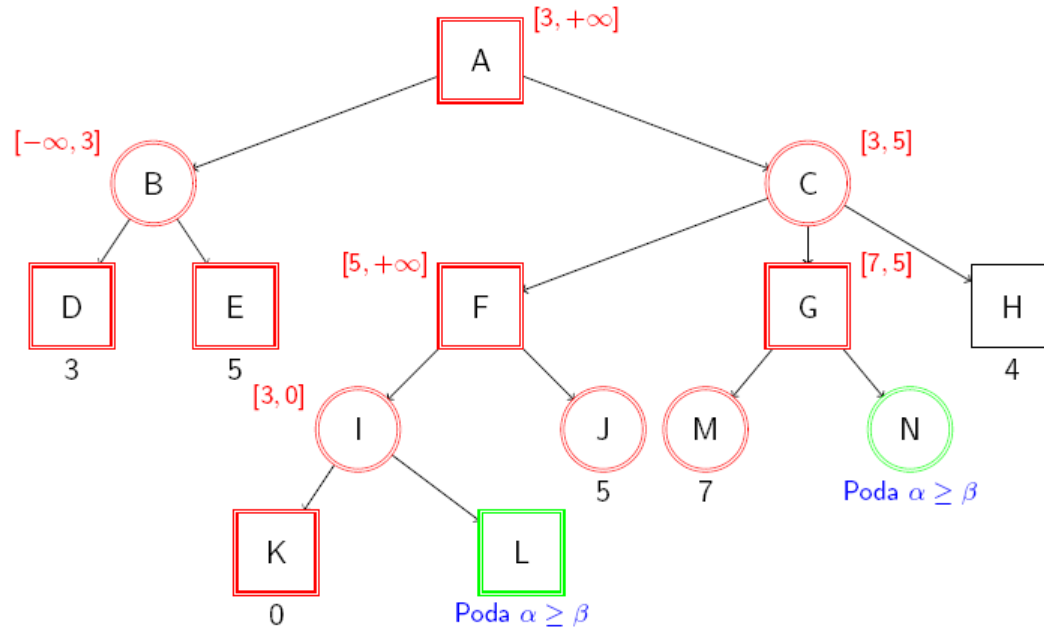
Ejemplo



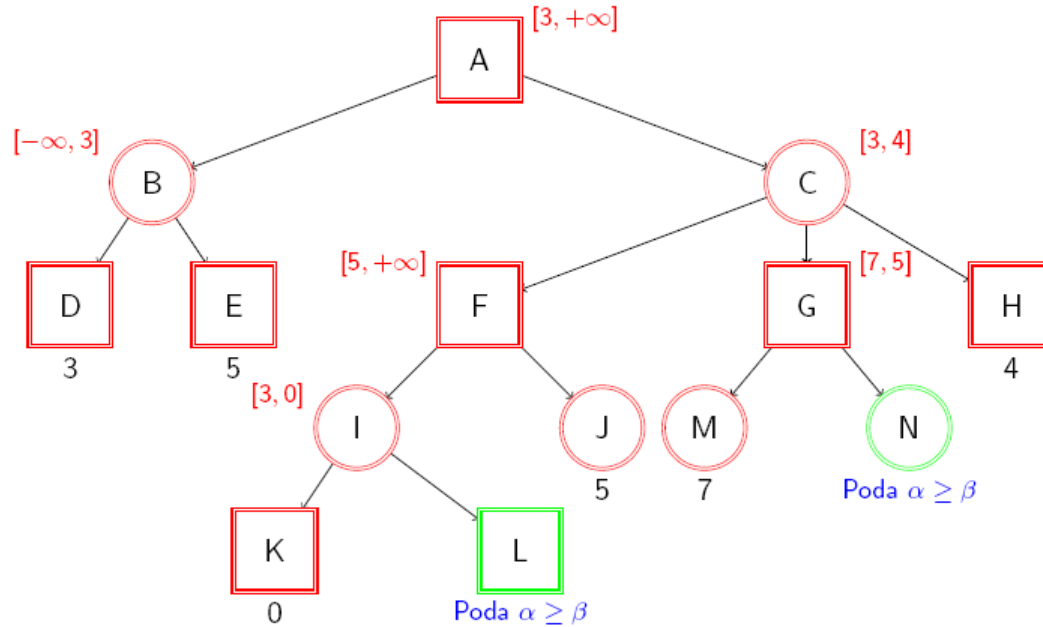
Ejemplo



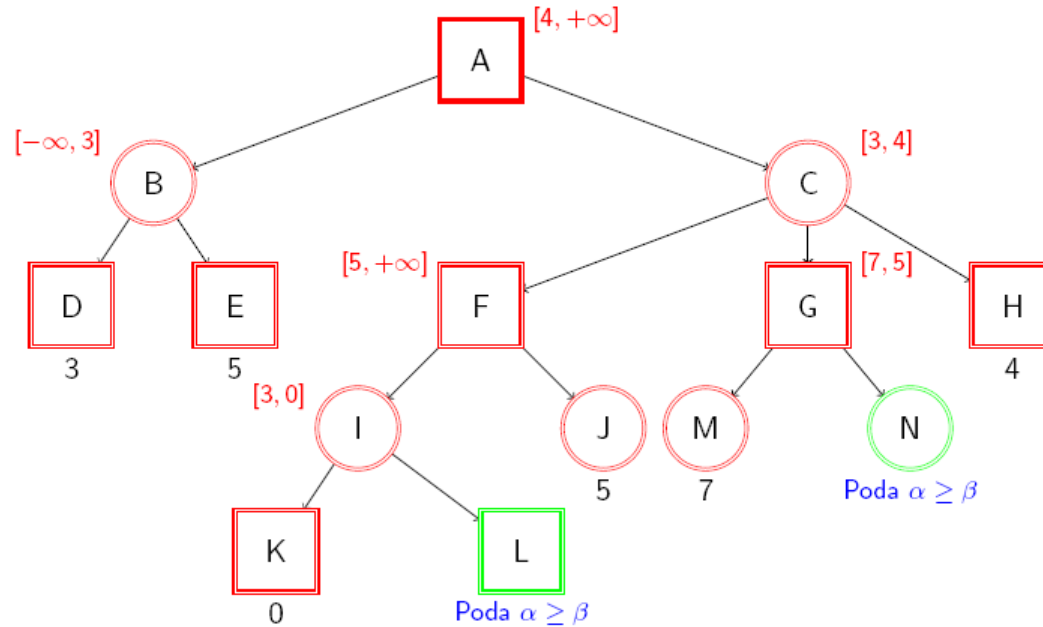
Ejemplo



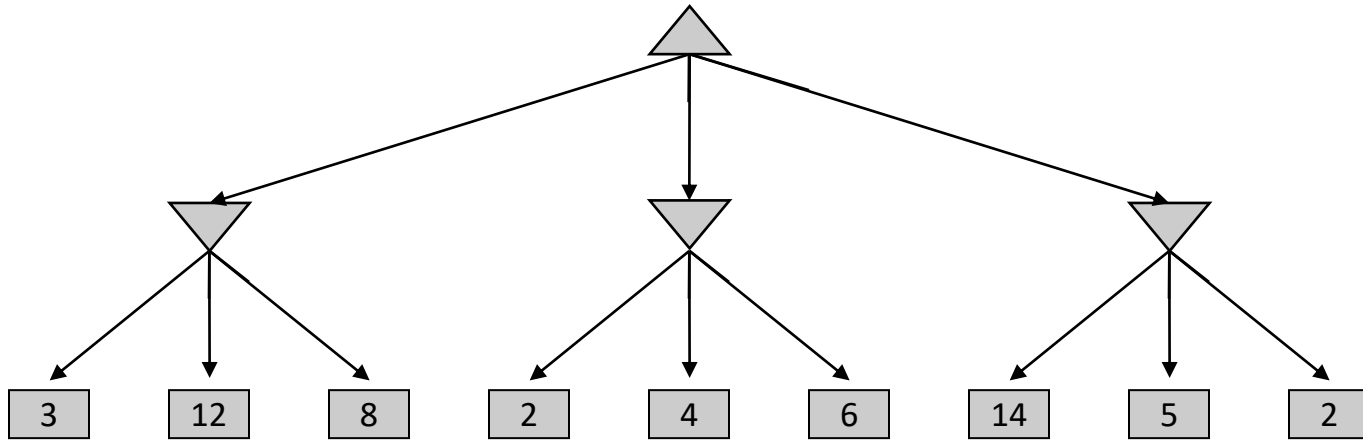
Ejemplo



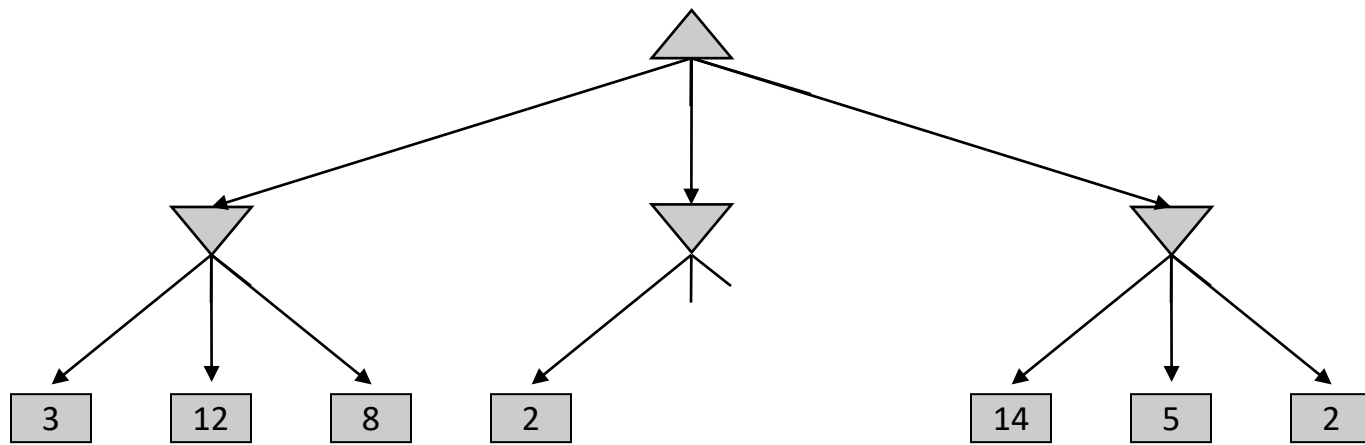
Ejemplo



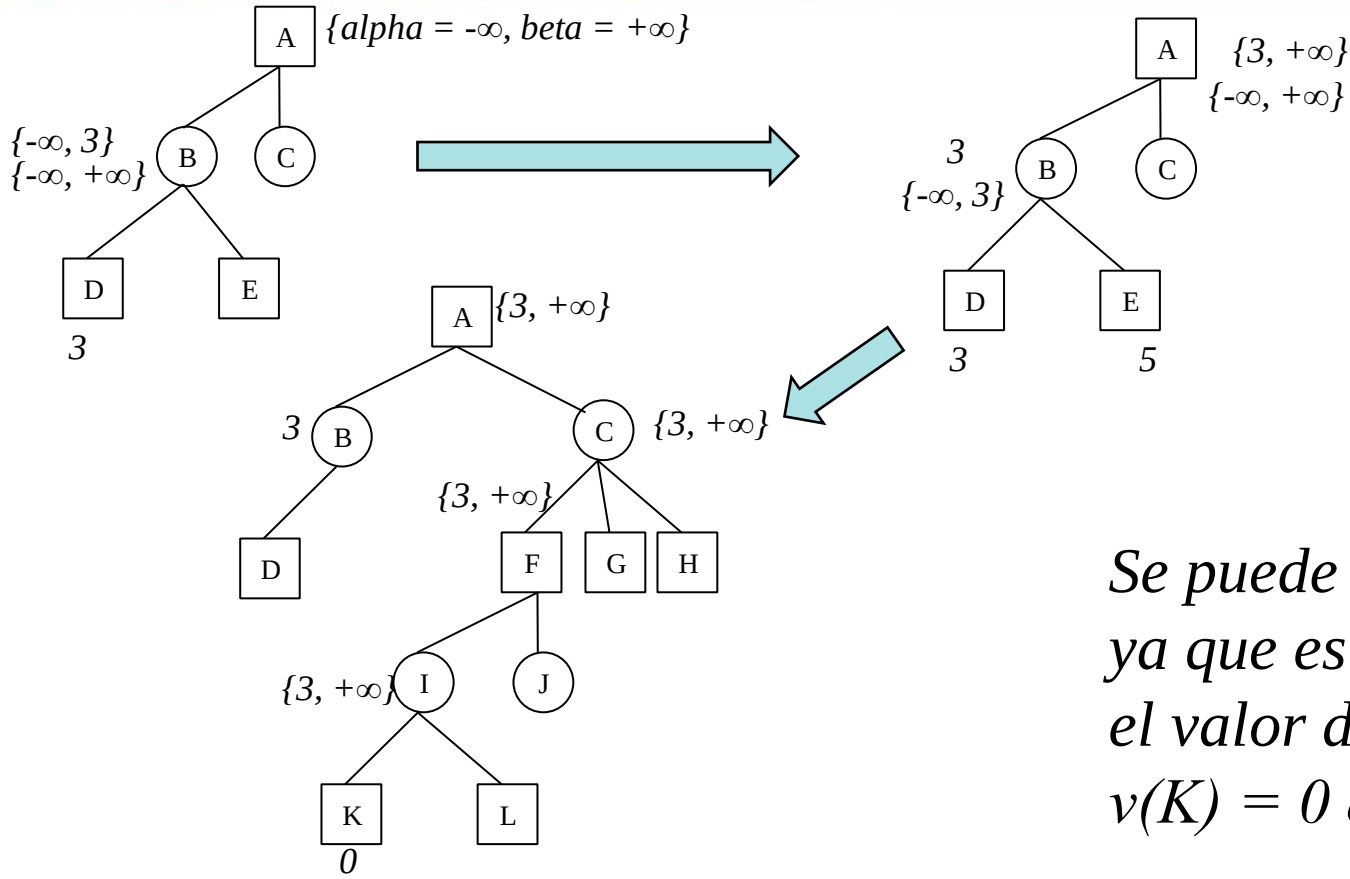
Poda alfa-beta



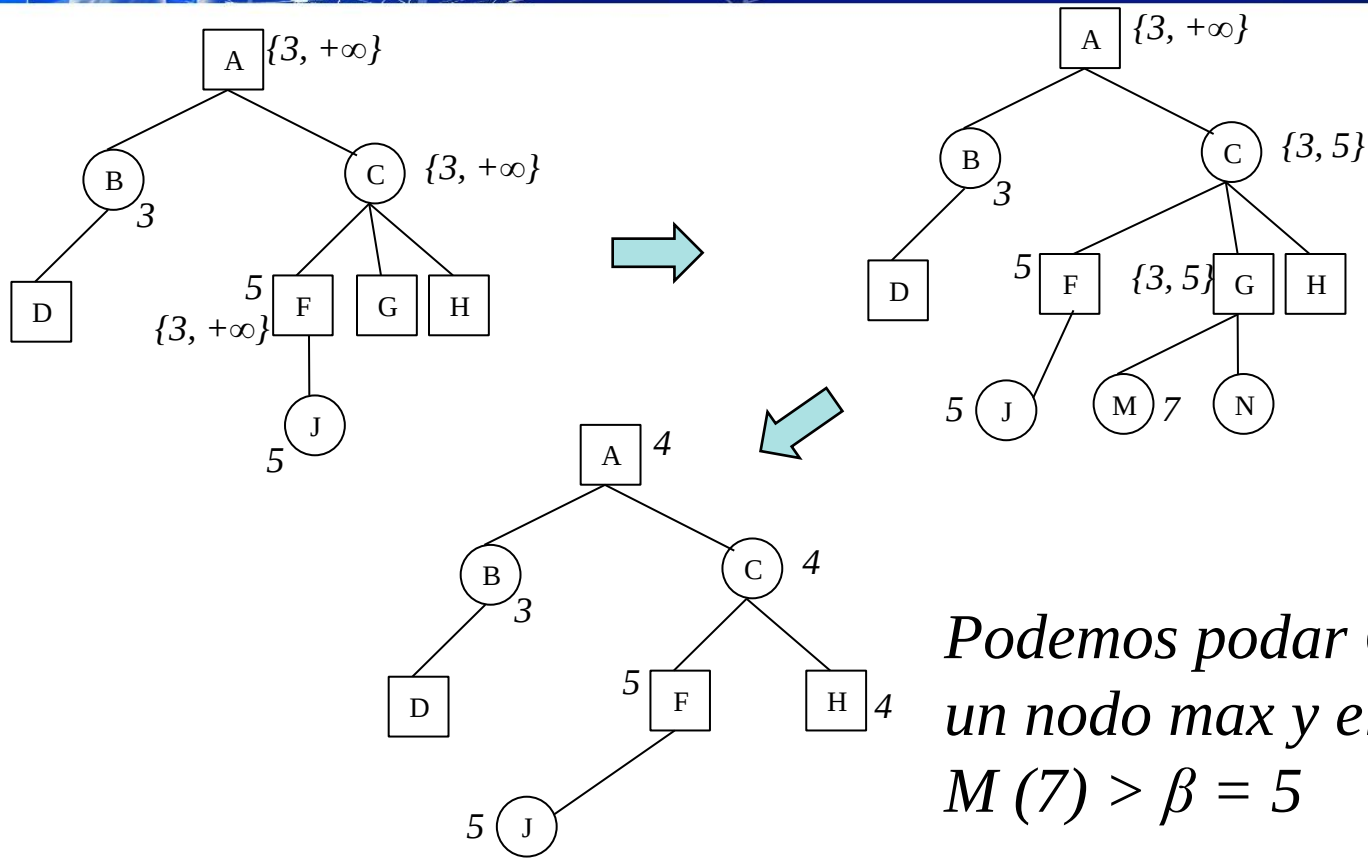
Poda alfa-beta





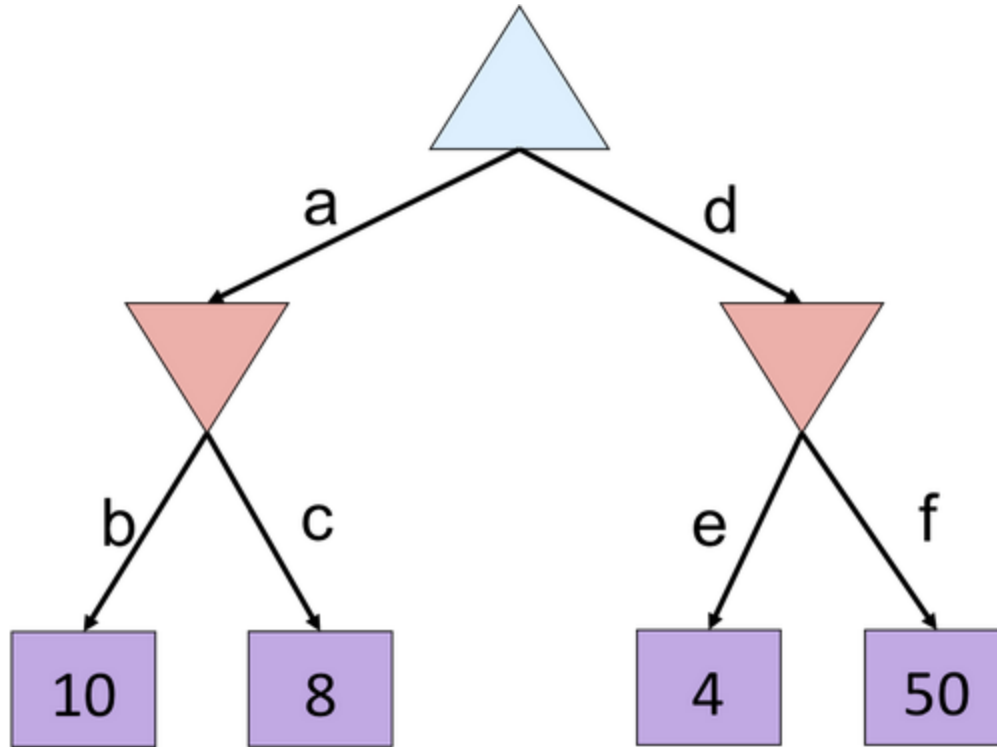


*Se puede podar I
ya que es un nodo min y
el valor de
 $v(K) = 0$ es $< \alpha = 3$*



Podemos podar G pues es un nodo max y el valor de $M(7) > \beta = 5$

Alpha-Beta Quiz



Alpha-Beta Quiz 2

